```
// ==================================================
// Software: N.E.K.O
// Version: V1.0
// ==================================================

// --------------------------------------------------
// FRONT 30 PAGES
// --------------------------------------------------


// ----------------------------------------

import { Tabs } from 'expo-router';
import React from 'react';

import { HapticTab } from '@/components/haptic-tab';
import { IconSymbol } from '@/components/ui/icon-symbol';
import { Colors } from '@/constants/theme';
import { useColorScheme } from '@/hooks/use-color-scheme';

export default function TabLayout() {
  const colorScheme = useColorScheme();

  return (
    <Tabs
      screenOptions={{
        tabBarActiveTintColor: Colors[colorScheme ?? 'light'].tint,
        headerShown: false,
        tabBarButton: HapticTab,
        // tabBarStyle: { display: 'none', height: 0 },
      }}>
      <Tabs.Screen
        name="index"
        options={{
          title: 'Home',
          tabBarIcon: ({ color }) => <IconSymbol size={28} name="house.fill" color={color} />,
        }}
      />
      <Tabs.Screen
        name="main"
        options={{
          title: 'Main UI',
          tabBarIcon: ({ color }) => <IconSymbol size={28} name="house.fill" color={color} />,
        }}
      />
    </Tabs>
  );
}

// ----------------------------------------

import { StyleSheet, View, TouchableOpacity, Text } from 'react-native';
import { SafeAreaView } from 'react-native-safe-area-context';
import { useRouter } from 'expo-router';
import { useEffect, useState } from 'react';
import { useIsFocused } from '@react-navigation/native';
import { useTranslation } from 'react-i18next';
import { useDevConnectionConfig } from '@/hooks/useDevConnectionConfig';
import { hasUserStoredConfig } from '@/services/DevConnectionStorage';
import { sessionStore } from '@/utils/sessionStore';
import { useColorScheme } from '@/hooks/use-color-scheme';
import { changeLanguage, SUPPORTED_LANGUAGES } from '@/i18n';

type ConnectionStatus = 'online' | 'offline';

// Status map will be populated by translation
const STATUS_COLORS: Record<ConnectionStatus, string> = {
  online:  '#40c5f1',
  offline: '#ff4d4d',
};

// ■■/■■■■■■■■■■■■■ theme.css ■ dark-mode.css■
const LIGHT = {
  container:       '#e3f4ff',
  card:            '#f0f8ff',
  cardBorder:      '#b3e5fc',
  actionBtn:       '#f0f8ff',
  actionBorder:    '#b3e5fc',
  configBtn:       '#f0f8ff',
  configBtnBorder: '#b3e5fc',
  textPrimary:     '#1a1a2e',
  textSub:         '#555',
  textMuted:       '#888',
  textOffline:     '#999',
```

```
  titleColor:     '#40c5f1',
  sectionTitle:   '#40c5f1',
  configBtnText:  '#40c5f1',
};

const DARK = {
  container:      '#000',
  card:           'rgba(30, 30, 30, 0.6)',
  cardBorder:     'rgba(255, 255, 255, 0.1)',
  actionBtn:      'rgba(64, 197, 241, 0.1)',
  actionBorder:   'rgba(64, 197, 241, 0.3)',
  configBtn:      'rgba(64, 197, 241, 0.08)',
  configBtnBorder: 'rgba(64, 197, 241, 0.2)',
  textPrimary:    '#fff',
  textSub:        '#888',
  textMuted:      '#666',
  textOffline:    '#555',
  titleColor:     '#40c5f1',
  sectionTitle:   '#40c5f1',
  configBtnText:  '#40c5f1',
};

export default function HomeScreen() {
  const router = useRouter();
  const { config, isLoaded, reload } = useDevConnectionConfig();
  const colorScheme = useColorScheme();
  const isDark = colorScheme === 'dark';
  const theme = isDark ? DARK : LIGHT;
  const { t, i18n } = useTranslation();

  const isFocused = useIsFocused();
  const [isConnected, setIsConnected] = useState(sessionStore.isConnected);
  const [isUserConfigured, setIsUserConfigured] = useState(false);

  // 订阅 WebSocket 连接状态变化
  useEffect(() => sessionStore.subscribe(setIsConnected), []);

  // 每次页面获得焦点时同步配置状态（扫码/手动配置后返回首页可立即更新）
  useEffect(() => {
    if (!isLoaded || !isFocused) return;
    hasUserStoredConfig().then(setIsUserConfigured);
    reload();
  }, [isLoaded, isFocused, reload]);

  const status: ConnectionStatus = isConnected ? 'online' : 'offline';
  const statusColor = STATUS_COLORS[status];
  const statusText = status === 'online' ? t('home.status.online') : t('home.status.offline');
  const showIp = isUserConfigured && isConnected;

  return (
    <SafeAreaView style={[styles.container, { backgroundColor: theme.container }]}>
      <View style={styles.content}>
        {/* 标题区域 */}
        <View style={styles.header}>
          <Text style={[styles.title, { color: theme.titleColor }]}>{t('home.title')}</Text>
        </View>

        {/* 快捷功能 */}
        <View style={styles.section}>
          <Text style={[styles.sectionTitle, { color: theme.sectionTitle }]}>{t('home.shortcuts')}</Text>
          <View style={styles.quickActions}>
            <TouchableOpacity
              style={[styles.actionButton, { backgroundColor: theme.actionBtn, borderColor: theme.actionBorder }]}
              onPress={() => router.push('/settings')}
              activeOpacity={0.8}
            >
              <Text style={styles.actionIcon}></Text>
              <Text style={[styles.actionText, { color: theme.textPrimary }]}>{t('home.apiSettings')}</Text>
            </TouchableOpacity>

            <TouchableOpacity
              style={[styles.actionButton, { backgroundColor: theme.actionBtn, borderColor: theme.actionBorder }]}
              onPress={() => router.push('/character-manager')}
              activeOpacity={0.8}
            >
              <Text style={styles.actionIcon}></Text>
              <Text style={[styles.actionText, { color: theme.textPrimary }]}>{t('home.characterManager')}</Text>
            </TouchableOpacity>
          </View>
        </View>

        {/* 语言选择 */}
        <View style={styles.section}>
          <Text style={[styles.sectionTitle, { color: theme.sectionTitle }]}> {t('settings.sections.language')}</Text>
```

```
          <View style={styles.langRow}>
            {SUPPORTED_LANGUAGES.map((lang) => (
              <TouchableOpacity
                key={lang.code}
                style={[
                  styles.langButton,
                  { backgroundColor: theme.actionBtn, borderColor: theme.actionBorder },
                  i18n.language === lang.code && { backgroundColor: theme.titleColor, borderColor: theme.titleColor },
                ]}
                onPress={() => changeLanguage(lang.code)}
                activeOpacity={0.8}
              >
                <Text style={[
                  styles.langText,
                  { color: theme.textPrimary },
                  i18n.language === lang.code && { color: '#fff', fontWeight: 'bold' },
                ]}>
                  {lang.name}
                </Text>
              </TouchableOpacity>
            ))}
          </View>
        </View>

        {/* 服务器配置 */}
        <View style={styles.section}>
          <Text style={[styles.sectionTitle, { color: theme.sectionTitle }]}>{t('home.serverConnection')}</Text>

          <View style={[styles.configCard, { backgroundColor: theme.card, borderColor: theme.cardBorder }]}>
            <View style={styles.configRow}>
              <Text style={[styles.configLabel, { color: theme.textSub }]}>{t('connection.status.connected')}</Text>
              <View style={styles.statusIndicator}>
                <View style={[styles.statusDot, { backgroundColor: statusColor }]} />
                <Text style={[styles.statusText, { color: statusColor }]}>{statusText}</Text>
              </View>
            </View>
            {showIp ? (
              <Text style={[styles.configValue, { color: theme.textPrimary }]}>
                {config.host}:{config.port}
              </Text>
            ) : isUserConfigured ? (
              <Text style={[styles.configValueOffline, { color: theme.textOffline }]}>{t('home.status.configured')}</Text>
            ) : (
              <Text style={[styles.configValueOffline, { color: theme.textOffline }]}>{t('home.status.unconfigured')}</Text>
            )}
            <Text style={[styles.configSubtext, { color: theme.textMuted }]}>{t('home.actions.currentRole')}: {config.characterName}</Text>
          </View>

          <View style={styles.configButtons}>
            <TouchableOpacity
              style={[styles.configButton, { backgroundColor: theme.configBtn, borderColor: theme.configBtnBorder }]}
              onPress={() => router.push('/server-config')}
              activeOpacity={0.8}
            >
              <Text style={styles.configButtonIcon}></Text>
              <Text style={[styles.configButtonText, { color: theme.configBtnText }]}>{t('home.actions.manualConfig')}</Text>
            </TouchableOpacity>

            <TouchableOpacity
              style={[styles.configButton, { backgroundColor: theme.configBtn, borderColor: theme.configBtnBorder }]}
              onPress={() => router.push({ pathname: '/qr-scanner', params: { returnTo: '/main' } })}
              activeOpacity={0.8}
            >
              <Text style={styles.configButtonIcon}></Text>
              <Text style={[styles.configButtonText, { color: theme.configBtnText }]}>{t('home.actions.qrConfig')}</Text>
            </TouchableOpacity>
          </View>
        </View>
      </View>
    </SafeAreaView>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    // backgroundColor 由 t.container 动态注入
  },
  content: {
    flex: 1,
    justifyContent: 'center',
  },
  header: {
    alignItems: 'center',
```

```
  paddingVertical: 40,
  paddingHorizontal: 20,
},
title: {
  fontSize: 32,
  fontWeight: 'bold',
  letterSpacing: 2,
},
section: {
  marginBottom: 28,
  paddingHorizontal: 20,
},
sectionTitle: {
  fontSize: 13,
  fontWeight: '600',
  marginBottom: 12,
  letterSpacing: 1,
  textTransform: 'uppercase',
},
quickActions: {
  flexDirection: 'row',
  gap: 12,
},
actionButton: {
  flex: 1,
  borderRadius: 16,
  paddingVertical: 24,
  paddingHorizontal: 16,
  alignItems: 'center',
  justifyContent: 'center',
  gap: 10,
  borderWidth: 1,
},
actionIcon: {
  fontSize: 28,
},
actionText: {
  fontSize: 14,
  fontWeight: '600',
},
langRow: {
  flexDirection: 'row',
  flexWrap: 'wrap',
  gap: 8,
},
langButton: {
  width: '31%',
  paddingHorizontal: 8,
  paddingVertical: 8,
  borderRadius: 10,
  borderWidth: 1,
  alignItems: 'center',
},
langText: {
  fontSize: 13,
  textAlign: 'center',
},
configCard: {
  borderRadius: 16,
  padding: 16,
  borderWidth: 1,
},
configRow: {
  flexDirection: 'row',
  justifyContent: 'space-between',
  alignItems: 'center',
  marginBottom: 12,
},
configLabel: {
  fontSize: 12,
},
statusIndicator: {
  flexDirection: 'row',
  alignItems: 'center',
  gap: 6,
},
statusDot: {
  width: 8,
  height: 8,
  borderRadius: 4,
},
statusText: {
  fontSize: 12,
  fontWeight: '600',
```

```
  },
  configValue: {
    fontSize: 20,
    fontFamily: 'monospace',
    fontWeight: '600',
  },
  configValueOffline: {
    fontSize: 14,
    fontStyle: 'italic',
  },
  configSubtext: {
    fontSize: 13,
    marginTop: 6,
  },
  configButtons: {
    flexDirection: 'row',
    gap: 12,
    marginTop: 12,
  },
  configButton: {
    flex: 1,
    borderRadius: 12,
    paddingVertical: 16,
    paddingHorizontal: 12,
    alignItems: 'center',
    justifyContent: 'center',
    gap: 6,
    borderWidth: 1,
  },
  configButtonIcon: {
    fontSize: 20,
  },
  configButtonText: {
    fontSize: 13,
    fontWeight: '600',
  },
});

// ----------------------------------------

import { createCharactersApiClient } from '@/services/api/characters';
import type { CharactersData } from '@/services/api/characters';
import { buildHttpBaseURL, appendP2PToken } from '@/utils/devConnectionConfig';
import { useAudio } from '@/hooks/useAudio';
import { useChatMessages } from '@/hooks/useChatMessages';
import { useDevConnectionConfig } from '@/hooks/useDevConnectionConfig';
import { useUdpP2PConnection } from '@/hooks/useUdpP2PConnection';
import { useLipSync } from '@/hooks/useLipSync';
import { useLive2D } from '@/hooks/useLive2D';
import { useLive2DAgentBackend } from '@/hooks/useLive2DAgentBackend';
import { useLive2DPreferences } from '@/hooks/useLive2DPreferences';
import { useImagePicker } from '@/hooks/useImagePicker';
import { useCamera } from '@/hooks/useCamera';
import { useCameraStream } from '@/hooks/useCameraStream';
import { CameraView } from 'expo-camera';
import { ImageMessageService } from '@/services/imageMessage';
import { mainManager } from '@/utils/MainManager';
import { sessionStore } from '@/utils/sessionStore';
import { VoicePrepareOverlay } from '@/components/VoicePrepareOverlay';
import { useFocusEffect } from '@react-navigation/native';
import { useLocalSearchParams } from 'expo-router';
import { useTranslation } from 'react-i18next';
import { useCallback, useEffect, useMemo, useRef, useState } from 'react';
import {
  ActivityIndicator,
  Alert, AppState, Appearance, Dimensions, Image, Modal, Platform, ScrollView,
  StyleSheet, Text, TextInput, TouchableOpacity, TouchableWithoutFeedback, View,
} from 'react-native';
import { ReactNativeLive2dView } from 'react-native-live2d';
import { Gesture, GestureDetector } from 'react-native-gesture-handler';
import {
  Live2DRightToolbar,
  ChatContainer,
  StatusToast,
  type Live2DRightToolbarPanel,
  type Live2DSettingsToggleId,
  type Live2DSettingsState,
  type Live2DAgentToggleId,
  type ConnectionStatus,
  type StatusToastHandle,
} from '@project_neko/components';

type MainUIScreenProps = {}
```

```
// 边界保护：逻辑视图范围为 ±1.0，设置为 0.9 确保模型始终大部分在屏幕内
const POSITION_LIMIT = 0.9;
const clampPos = (v: number) => Math.max(-POSITION_LIMIT, Math.min(POSITION_LIMIT, v));

// 缩放范围限制
const SCALE_MIN = 0.3;
const SCALE_MAX = 2.0;
const clampScale = (v: number) => Math.max(SCALE_MIN, Math.min(SCALE_MAX, v));

// 生成消息 ID
function generateMessageId(counter: number): string {
  return `msg-${Date.now()}-${counter}`;
}

const MainUIScreen: React.FC<MainUIScreenProps> = () => {
  const { t } = useTranslation();

  const [isPageFocused, setIsPageFocused] = useState(true);

  // 角色选择 Modal 状态
  const [characterModalVisible, setCharacterModalVisible] = useState(false);
  const [voiceBlockModalVisible, setVoiceBlockModalVisible] = useState(false);
  const [characterList, setCharacterList] = useState<string[]>([]);
  const [currentCatgirl, setCurrentCatgirl] = useState<string | null>(null);
  const [characterLoading, setCharacterLoading] = useState(false);
  const [switchedCharacterName, setSwitchedCharacterName] = useState<string | null>(null);
  const [switchError, setSwitchError] = useState<string | null>(null);
  const switchedNameTimerRef = useRef<ReturnType<typeof setTimeout> | null>(null);
  const switchErrorTimerRef = useRef<ReturnType<typeof setTimeout> | null>(null);
  const characterLoadingTimerRef = useRef<ReturnType<typeof setTimeout> | null>(null);
  const [isChatForceCollapsed, setIsChatForceCollapsed] = useState(false);
  const [voicePrepareStatus, setVoicePrepareStatus] = useState<'preparing' | 'ready' | null>(null);
  const isSwitchingCharacterRef = useRef(false);
  //  新增：应用是否在后台的标志 ref，用于在拍照等场景忽略 WebSocket 错误
  const isInBackgroundRef = useRef(false);
  //  新增：记录是否是从后台恢复，用于显示"已恢复连接"提示
  const wasInBackgroundRef = useRef(false);
  // AppState 后台延迟重置 timer ref（确保组件卸载时清理）
  const appStateTimerRef = useRef<ReturnType<typeof setTimeout> | null>(null);
  // StatusToast ref，用于显示连接状态提示
  const statusToastRef = useRef<StatusToastHandle>(null);
  //  新增：调试信息面板
  const [debugPanelVisible, setDebugPanelVisible] = useState(false);
  // 合并为单一对象，确保 modelName 和 modelUrl 同步更新，避免两次 setState 触发两次 useLive2D effect
  const [live2dModel, setLive2dModel] = useState<{ name: string; url: string | undefined; itemId?: string }>({
    name: 'mao_pro',
    url: undefined,
  });
  // ref 持有最新 currentCatgirl，供 onConnectionChange 闭包安全读取（避免 stale closure）
  const currentCatgirlRef = useRef<string | null>(null);
  currentCatgirlRef.current = currentCatgirl;
  // ref 持有最新值，供 useFocusEffect 闭包读取（避免 stale closure）
  const live2dModelRef = useRef(live2dModel);
  live2dModelRef.current = live2dModel;
  const { config, isLoaded: isConfigLoaded, setConfig, applyQrRaw, refreshFromCloud } = useDevConnectionConfig();

  // UDP P2P 连接（自动尝试三层回退）
  const udpConnection = useUdpP2PConnection(
    config,
    isConfigLoaded,
    setConfig,
    refreshFromCloud
  );

  const params = useLocalSearchParams<{
    qr?: string;
    host?: string;
    port?: string;
    name?: string;
    characterName?: string;
  }>();
  const lastAppliedQrRef = useRef<string | null>(null);

  //  监听应用状态变化，在应用进入后台时标记状态（如拍照场景）
  useEffect(() => {
    const subscription = AppState.addEventListener('change', (nextAppState) => {
      if (nextAppState === 'background' || nextAppState === 'inactive') {
        console.log(' 应用进入后台，标记后台状态');
        if (appStateTimerRef.current) clearTimeout(appStateTimerRef.current);
        isInBackgroundRef.current = true;
      } else if (nextAppState === 'active') {
        console.log(' 应用回到前台，延迟重置后台状态');
        // 标记是从后台恢复，用于显示"已恢复连接"提示
        wasInBackgroundRef.current = true;
```

```
      // 延迟重置，给 WebSocket 重连时间，避免显示错误
      if (appStateTimerRef.current) clearTimeout(appStateTimerRef.current);
      appStateTimerRef.current = setTimeout(() => {
        appStateTimerRef.current = null;
        isInBackgroundRef.current = false;
        wasInBackgroundRef.current = false;
        console.log(' 后台状态标志已重置');
      }, 2000);
    }
  });

  return () => {
    subscription.remove();
    if (appStateTimerRef.current) {
      clearTimeout(appStateTimerRef.current);
      appStateTimerRef.current = null;
    }
  };
}, []);

useEffect(() => {
  const qrParam = typeof params.qr === 'string' ? params.qr : undefined;
  let raw: string | null = null;

  if (qrParam) {
    try {
      raw = decodeURIComponent(qrParam);
    } catch {
      raw = qrParam;
    }
  } else {
    const host = typeof params.host === 'string' ? params.host.trim() : '';
    const portStr = typeof params.port === 'string' ? params.port.trim() : '';
    const name =
      typeof params.characterName === 'string'
        ? params.characterName.trim()
        : typeof params.name === 'string'
          ? params.name.trim()
          : '';

    if (host || portStr || name) {
      const payload: { host?: string; port?: number; characterName?: string } = {};
      if (host) payload.host = host;
      if (portStr && /^\d+$/.test(portStr)) payload.port = Number(portStr);
      if (name) payload.characterName = name;
      raw = JSON.stringify(payload);
    }
  }

  if (!raw) return;
  if (lastAppliedQrRef.current === raw) return;
  lastAppliedQrRef.current = raw;

  applyQrRaw(raw).then((res) => {
    if (!res.ok) {
      Alert.alert(t('qrScanner.invalidCode'), res.error);
    }
  });
}, [applyQrRaw, params.characterName, params.host, params.name, params.port, params.qr]);

// 从后端获取角色对应的 Live2D 模型信息并更新状态
const syncLive2dModel = useCallback(async (catgirlName: string) => {
  console.log(' [syncLive2dModel] called, catgirlName =', catgirlName, 'isConfigLoaded =', isConfigLoaded);
  // 等待配置加载完成
  if (!isConfigLoaded) return;

  try {
    const apiBase = buildHttpBaseURL(config);
    const client = createCharactersApiClient(apiBase, config.p2p?.token);
    const modelRes = await client.getCurrentLive2dModel(catgirlName);
    if (__DEV__) console.log(' [syncLive2dModel] API 返回:', JSON.stringify(modelRes));
    if (modelRes.success && modelRes.model_info) {
      const modelUrl = appendP2PToken(`${apiBase}${modelRes.model_info.path}`, config.p2p?.token);
      if (__DEV__) console.log(' [syncLive2dModel] 设置模型 URL:', modelUrl);
      setLive2dModel({
        name: modelRes.model_info.name,
        url: modelUrl,
        itemId: modelRes.model_info.item_id,
      });
    } else {
      console.warn(' [syncLive2dModel] API 返回但无 model_info:', modelRes);
    }
  } catch (e) {
    console.warn('[syncLive2dModel] 获取模型信息失败:', e);
```

```
  }
}, [config, isConfigLoaded]); // eslint-disable-line react-hooks/exhaustive-deps

// 启动时从服务端获取当前角色，以服务端为准
useEffect(() => {
  if (__DEV__) console.log(' [syncCurrentCatgirl] isConfigLoaded =', isConfigLoaded, 'config =', JSON.stringify({ host: config.host, port: con
fig.port, char: config.characterName, hasP2P: !!config.p2p }));
  // 等待配置加载完成
  if (!isConfigLoaded) return;
  console.log(' [syncCurrentCatgirl] 配置已加载，开始同步角色');

  const syncCurrentCatgirl = async () => {
    try {
      const apiBase = buildHttpBaseURL(config);
      if (__DEV__) console.log(' [syncCurrentCatgirl] apiBase =', apiBase);
      const client = createCharactersApiClient(apiBase, config.p2p?.token);
      const res = await client.getCurrentCatgirl();
      if (__DEV__) console.log(' [syncCurrentCatgirl] getCurrentCatgirl 返回:', JSON.stringify(res));
      if (res.current_catgirl) {
        setCurrentCatgirl(res.current_catgirl);
        if (config.characterName !== res.current_catgirl) {
          await setConfig({ ...config, characterName: res.current_catgirl });
        }
        console.log(' [syncCurrentCatgirl] 准备 syncLive2dModel:', res.current_catgirl);
        await syncLive2dModel(res.current_catgirl);
        console.log(' [syncCurrentCatgirl] syncLive2dModel 完成');

        // 发送 start_session 以同步角色音色
        // 使用 audio.isReadyRef（ref，始终最新）避免闭包捕获 isConnected 旧值
        setTimeout(() => {
          if (audio.isReadyRef.current) {
            console.log(' 发送 start_session 以同步角色音色');
            audio.sendMessage({
              action: 'start_session',
              input_type: 'text',
              audio_format: 'PCM_48000HZ_MONO_16BIT',
              new_session: false,
            });
          }
        }, 500);
      }
    } catch (e) {
      console.warn(' [syncCurrentCatgirl] 失败:', e);
      // 网络不通时降级：用本地缓存初始化 UI
      if (config.characterName) setCurrentCatgirl(config.characterName);
    }
  };
  syncCurrentCatgirl();
}, [isConfigLoaded]); // eslint-disable-line react-hooks/exhaustive-deps

// 工具栏状态管理（与 Web 版本一致）
const [isMobile, setIsMobile] = useState(true); // RN 默认为移动端
const [screenHeight, setScreenHeight] = useState(() => Dimensions.get('window').height);
const [toolbarGoodbyeMode, setToolbarGoodbyeMode] = useState(false);
const [toolbarMicEnabled, setToolbarMicEnabled] = useState(false);
const [toolbarOpenPanel, setToolbarOpenPanel] = useState<Live2DRightToolbarPanel>(null);
const [toolbarSettings, setToolbarSettings] = useState<Live2DSettingsState>({
  mergeMessages: true,
  allowInterrupt: true,
  proactiveChat: false,
  proactiveVision: false,
  darkMode: false,
});

// 消息去重：跟踪已发送消息的 clientMessageId（使用 Map 存储时间戳，支持 TTL 清理）
// 配置：TTL 5分钟，最大条目数 1000，清理间隔 1分钟
const DEDUPE_TTL_MS = 5 * 60 * 1000;
const DEDUPE_MAX_SIZE = 1000;
const DEDUPE_CLEANUP_INTERVAL_MS = 60 * 1000;
const sentClientMessageIds = useRef<Map<string, number>>(new Map());
const messageCounterRef = useRef(0);

// 定期清理过期的去重条目，防止内存无限增长
useEffect(() => {
  const cleanup = () => {
    const now = Date.now();
    const map = sentClientMessageIds.current;

    // 删除超过 TTL 的条目
    for (const [id, timestamp] of map) {
      if (now - timestamp > DEDUPE_TTL_MS) {
        map.delete(id);
      }
    }
```

```
    // 如果仍超过最大数量，按时间戳淘汰最旧的条目
    if (map.size > DEDUPE_MAX_SIZE) {
      const entries = Array.from(map.entries()).sort((a, b) => a[1] - b[1]);
      const toRemove = entries.slice(0, map.size - DEDUPE_MAX_SIZE);
      for (const [id] of toRemove) {
        map.delete(id);
      }
    }
  };

  const interval = setInterval(cleanup, DEDUPE_CLEANUP_INTERVAL_MS);
  return () => clearInterval(interval);
}, []);

// Text session 管理（与 Web 端一致）
const [isTextSessionActive, setIsTextSessionActive] = useState(false);
const sessionStartedResolverRef = useRef<((value: boolean) => void) | null>(null);
const sessionTimeoutRef = useRef<ReturnType<typeof setTimeout> | null>(null);
const pendingSessionPromiseRef = useRef<Promise<boolean> | null>(null);

// Agent Backend 管理（传入 openPanel 以支持动态刷新）
const { agent, onAgentChange, refreshAgentState } = useLive2DAgentBackend({
  apiBase: `http://${config.host}:${config.port}`,
  showToast: (message, duration) => {
    Alert.alert(t('common.alert'), message);
  },
  openPanel: toolbarOpenPanel === 'agent' ? 'agent' : null,
});

// Live2D Preferences 持久化
const { repository: preferencesRepository } = useLive2DPreferences();

const chat = useChatMessages({
  maxMessages: 100,
});

// 图片选择器 hook
const imagePicker = useImagePicker({
  maxSelectionCount: 5,
  allowsMultipleSelection: true,
});

// 相机 hook
const camera = useCamera();

// 相机拍照结果状态（传递给 ChatContainer 的 externalPendingImages）
const [cameraPendingImages, setCameraPendingImages] = useState<{ id: string; base64: string }[]>([]);

// 处理相机拍照结果
useEffect(() => {
  if (camera.photo) {
    // 将拍照结果添加到待发送列表
    const newImage = {
      id: `camera-${Date.now()}`,
      base64: camera.photo.base64, // 纯 base64，不添加前缀
    };
    setCameraPendingImages(prev => [...prev, newImage].slice(0, 5));
    camera.clearPhoto();
  }
}, [camera.photo, camera.clearPhoto]);

// 稳定 P2P 配置引用，避免不必要的重连（依赖整个 p2p 对象，而非只追踪 token）
const p2pConfig = useMemo(() => config.p2p, [config.p2p]);

const audio = useAudio({
  host: config.host,
  port: config.port,
  characterName: config.characterName,
  p2p: p2pConfig,  // P2P 配置（如果存在则使用 P2P 模式连接）
  enabled: isConfigLoaded,  // 等待配置加载完成后再连接
  isSwitchingRef: isSwitchingCharacterRef,  // 传入角色切换标志，用于在切换期间忽略错误
  isInBackgroundRef: isInBackgroundRef,  // 传入后台标志，用于在拍照等场景忽略错误
  onMessage: async (event) => {
    // 二进制音频数据已由 @project_neko/audio-service 自动播放（通过 Realtime binary 事件接管）
    // 这里仅保留文本消息处理逻辑
    if (typeof event.data !== 'string') return;

    // 检查 clientMessageId 用于去重
    let parsedMsg: any = null;
    try {
      parsedMsg = JSON.parse(event.data);
      const clientMessageId = parsedMsg?.clientMessageId as string | undefined;
      if (clientMessageId && sentClientMessageIds.current.has(clientMessageId)) {
```

```
    // 服务器回显，跳过处理
    sentClientMessageIds.current.delete(clientMessageId);
    return;
  }
} catch {
  // 非 JSON 消息，继续处理
}

// 处理 session_started 事件（text session 管理）
if (parsedMsg?.type === 'session_started') {
  const inputMode = parsedMsg.input_mode as string | undefined;
  console.log(' 收到 session_started，input_mode:', inputMode);
  if (inputMode === 'text') {
    setIsTextSessionActive(true);
  } else if (inputMode === 'audio') {
    // audio session 启动意味着 text session 已被替换，重置状态
    setIsTextSessionActive(false);
  }
  if (sessionTimeoutRef.current) {
    clearTimeout(sessionTimeoutRef.current);
    sessionTimeoutRef.current = null;
  }
  if (sessionStartedResolverRef.current) {
    sessionStartedResolverRef.current(true);
    sessionStartedResolverRef.current = null;
  }
  pendingSessionPromiseRef.current = null;
  return;
}

// 处理 session_failed 事件
if (parsedMsg?.type === 'session_failed') {
  console.log(' 收到 session_failed，input_mode:', parsedMsg.input_mode);
  setIsTextSessionActive(false);
  if (sessionTimeoutRef.current) {
    clearTimeout(sessionTimeoutRef.current);
    sessionTimeoutRef.current = null;
  }
  if (sessionStartedResolverRef.current) {
    sessionStartedResolverRef.current(false);
    sessionStartedResolverRef.current = null;
  }
  pendingSessionPromiseRef.current = null;
  return;
}

// 处理 session_ended_by_server 事件（服务端主动终止 session，如 API 断连）
if (parsedMsg?.type === 'session_ended_by_server') {
  console.log(' 收到 session_ended_by_server，input_mode:', parsedMsg.input_mode);
  setIsTextSessionActive(false);
  return;
}

// 处理文本消息并通过 MainManager 协调
const result = await chat.handleWebSocketMessage(event);

// 根据消息类型，通过 MainManager 触发相应的行为
if (result?.type === 'gemini_response') {
  mainManager.onGeminiResponse(result.isNewMessage);
} else if (result?.type === 'user_activity') {
  mainManager.onUserSpeechDetected();
} else if (result?.type === 'turn_end') {
  mainManager.onTurnEnd(result.fullText);
} else if ((result?.type === 'status' || result?.type === 'system_notice') && result.message) {
  // 系统状态消息通过 Toast 显示，不加入聊天列表
  statusToastRef.current?.show(result.message, 3000);
} else if (result?.type === 'catgirl_switched' && result.characterName) {
  // 幂等保护：如果已在切换中且目标角色相同，跳过重复处理
  if (isSwitchingCharacterRef.current && currentCatgirlRef.current === result.characterName) {
    console.log(' [catgirl_switched] 已在切换中，跳过重复处理');
    return;
  }

  // 检查是否需要触发 WebSocket 重连
  const needsReconnect = config.characterName !== result.characterName;

  // 本地和远端切换统一由此处驱动
  // 立即停止旧角色的音频播放，防止切换后还听到旧角色的声音
  audio.clearAudioQueue();
  setIsChatForceCollapsed(true);
  setCharacterLoading(true);
  isSwitchingCharacterRef.current = true;
  setCurrentCatgirl(result.characterName);
  // 角色切换时重置 text session 状态，确保下次发送消息时重新初始化 session
```

```
    setIsTextSessionActive(false);
    await setConfig({ ...config, characterName: result.characterName });
    await syncLive2dModel(result.characterName);

    // 如果 config.characterName 已经等于新角色名，useAudio effect 不会重新运行
    // 需要手动发送 start_session 并完成切换
    if (!needsReconnect) {
      console.log(' [catgirl_switched] config 未变化，手动完成切换');
      // 清除 handleSwitchCharacter 设置的超时 timer
      if (characterLoadingTimerRef.current) {
        clearTimeout(characterLoadingTimerRef.current);
        characterLoadingTimerRef.current = null;
      }
      // 直接发送 start_session
      audio.sendMessage({
        action: 'start_session',
        input_type: 'text',
        audio_format: 'PCM_48000HZ_MONO_16BIT',
        new_session: false,
      });
      // 立即完成切换
      isSwitchingCharacterRef.current = false;
      setCharacterLoading(false);
      setIsChatForceCollapsed(false);
      // 清除之前的错误提示（如果有）
      setSwitchError(null);
      if (switchErrorTimerRef.current) {
        clearTimeout(switchErrorTimerRef.current);
        switchErrorTimerRef.current = null;
      }
      setSwitchedCharacterName(result.characterName);
      if (switchedNameTimerRef.current) clearTimeout(switchedNameTimerRef.current);
      switchedNameTimerRef.current = setTimeout(() => setSwitchedCharacterName(null), 2500);
      return;
    }

    // 竞态保护：syncLive2dModel 期间 onConnectionChange(true) 可能已经完成切换
    // 此时 isSwitchingCharacterRef 已被重置为 false，无需再设超时
    if (!isSwitchingCharacterRef.current) {
      console.log(' [catgirl_switched] 切换已在 syncLive2dModel 期间完成，跳过超时设置');
      return;
    }

    // 超时保护：15 秒内若未收到 onConnectionChange(true)，自动解除所有切换状态
    // 仅在 timer 未设置时才设置，避免覆盖 handleSwitchCharacter 设置的 timer
    if (!characterLoadingTimerRef.current) {
      characterLoadingTimerRef.current = setTimeout(() => {
        setCharacterLoading(false);
        setIsChatForceCollapsed(false);
        isSwitchingCharacterRef.current = false;
        characterLoadingTimerRef.current = null;
        setSwitchError('连接超时，角色切换失败');
        if (switchErrorTimerRef.current) clearTimeout(switchErrorTimerRef.current);
        switchErrorTimerRef.current = setTimeout(() => setSwitchError(null), 3000);
      }, 15000);
    }
  }
},
onConnectionChange: (connected) => {
  sessionStore.set(connected);
  if (connected) {
    // 连接成功，显示 Toast 提示
    if (wasInBackgroundRef.current) {
      // 从后台恢复后的重连
      statusToastRef.current?.show('已恢复连接', 2000);
    } else if (!isSwitchingCharacterRef.current) {
      // 普通连接成功（非角色切换）
      statusToastRef.current?.show('已连接到服务器', 2000);
    }
    if (isSwitchingCharacterRef.current) {
      // 发送 start_session 以重新加载角色音色
      console.log(' 发送 start_session 以重新加载角色音色');
      audio.sendMessage({
        action: 'start_session',
        input_type: 'text',
        audio_format: 'PCM_48000HZ_MONO_16BIT',
        new_session: false,
      });
      console.log(' start_session 已调用');

      // 立即重置角色切换标志，避免后续消息重复触发超时
      isSwitchingCharacterRef.current = false;
      console.log(' 角色切换标志已重置');
      setCharacterLoading(false);
```

```
      setIsChatForceCollapsed(false);
      // 清除超时保护 timer
      if (characterLoadingTimerRef.current) {
        clearTimeout(characterLoadingTimerRef.current);
        characterLoadingTimerRef.current = null;
      }
      // 清除之前的错误提示（如果有）
      setSwitchError(null);
      if (switchErrorTimerRef.current) {
        clearTimeout(switchErrorTimerRef.current);
        switchErrorTimerRef.current = null;
      }
      const name = currentCatgirlRef.current;
      setSwitchedCharacterName(name);
      if (switchedNameTimerRef.current) clearTimeout(switchedNameTimerRef.current);
      switchedNameTimerRef.current = setTimeout(() => setSwitchedCharacterName(null), 2500);
    }
  } else {
    // 连接断开，显示 Toast 提示（仅在非后台状态下显示）
    if (!isInBackgroundRef.current) {
      statusToastRef.current?.show('连接已断开，正在尝试重连...', 3000);
    }
    // 连接断开时重置 text session 状态
    setIsTextSessionActive(false);
  }
 }
});

// 摄像头流式 hook
const cameraStream = useCameraStream({
  sendMessage: audio.sendMessage,
  isConnected: audio.isConnected,
  isInBackgroundRef,
});

// 监听摄像头流错误
useEffect(() => {
  if (cameraStream.error) {
    statusToastRef.current?.show(`摄像头错误: ${cameraStream.error}`, 3000);
  }
}, [cameraStream.error]);

// 将 audio.connectionStatus 映射到 ConnectionStatus 类型
// 在角色切换期间，保持 'open' 状态，避免显示断开错误
const connectionStatus: ConnectionStatus = isSwitchingCharacterRef.current
  ? 'open'
  : (audio.isConnected ? 'open' : 'closed');

const live2d = useLive2D({
  modelName: live2dModel.name,
  modelUrl: live2dModel.url,
  modelItemId: live2dModel.itemId,
  backendHost: config.host,
  backendPort: config.port,
  // 由页面 focus 生命周期触发加载；避免 autoLoad + focus 双重触发导致重复加载
  autoLoad: false,
  // TODO: 集成 preferences repository 到 useLive2D hook
  // 这需要修改 useLive2D 以支持持久化
});

// 口型同步 hook（无平滑模式，与 Web 版本一致）
const lipSync = useLipSync({
  minAmplitude: 0.005,   // 最小振幅阈值（降低以更敏感）
  amplitudeScale: 1.0,   // 振幅缩放（调整嘴巴张开幅度）
  autoStart: false,      // 不自动启动，等待模型加载完成
});

useFocusEffect(
  useCallback(() => {
    console.log('Live2D页面获得焦点');

    // 设置页面为焦点状态
    setIsPageFocused(true);

    // 只有 url 已就绪（syncLive2dModel 完成后）才触发加载
    // 避免启动时 url 还是 undefined，回退到自拼 URL 加载错误模型
    if (live2dModelRef.current.url) {
      live2d.loadModel();
    }

    return () => {
      console.log('Live2D页面失去焦点');
      // 停止口型同步（stop 应为幂等；避免把 isActive 放进依赖导致 focus effect 重跑）
      lipSync.stop();
```

```
      console.log(' 口型同步已停止（页面失焦）');

      // 设置页面为失去焦点状态
      setIsPageFocused(false);
      // 页面失去焦点时，重置模型状态，避免在重新获得焦点时立即加载模型
      // 这样可以确保 CubismFramework 有足够时间初始化
      // 注意：原生视图会在 onDetachedFromWindow 中自动清理资源
      live2d.unloadModel();
    };
  }, [live2d.loadModel, live2d.unloadModel, lipSync.stop])
);

// 角色切换后 modelUrl 变化时，页面已聚焦无法靠 useFocusEffect 触发，需单独监听
// 注意：live2dModel 变化时 useLive2D 已因 modelUrl 变化重建 service（新 service 天然干净）
// 不需要 unloadModel，直接 loadModel 即可；用 ref 确保拿到新 service 的最新引用
const loadModelRef = useRef(live2d.loadModel);
loadModelRef.current = live2d.loadModel;
useEffect(() => {
  if (!isPageFocused || !live2dModel.url) return;
  const timer = setTimeout(() => {
    loadModelRef.current();
  }, 0);
  return () => clearTimeout(timer);
}, [live2dModel]); // eslint-disable-line react-hooks/exhaustive-deps

// ===== 初始化 MainManager =====
useEffect(() => {
  console.log(' 主界面初始化');

  mainManager.init();

  if (audio.audioService) {
    mainManager.registerAudioService(audio.audioService);
  }

  if (live2d.live2dService) {
    mainManager.registerLive2DService(live2d.live2dService);
  }

  return () => {
    console.log(' 主界面清理');
  };
}, [audio.audioService, live2d.live2dService]);

useEffect(() => {
  console.log('live2d.live2dProps', live2d.live2dProps);
}, [live2d.live2dProps]);

useEffect(() => {
  console.log('live2d.modelState', live2d.modelState);
}, [live2d.modelState]);

// 监听模型状态，自动启动/停止口型同步
useEffect(() => {
  const jsReady = live2d.modelState.isReady && !!live2d.modelState.path;
  const nativeReady = live2d.isNativeModelLoaded;
  const shouldRun = isPageFocused && jsReady && nativeReady;

  if (shouldRun) {
    if (!lipSync.isActive) {
      console.log(' Live2D JS/Native 已就绪，启动口型同步');
      lipSync.start();
      console.log(' 口型同步已启动');
    }
    return;
  }

  if (lipSync.isActive) {
    console.log(' Live2D 未就绪或页面失焦，停止口型同步');
    lipSync.stop();
    console.log(' 口型同步已停止');
  }
}, [
  isPageFocused,
  live2d.modelState.isReady,
  live2d.modelState.path,
  live2d.isNativeModelLoaded,
  lipSync.isActive,
  lipSync.start,
  lipSync.stop,
]);

const handleLoadModel = useCallback(() => {
  live2d.loadModel();
```

```
}, [live2d.loadModel]);

const handleRecordingToggle = useCallback(() => {
  mainManager.toggleRecording();
}, []);

const handleLive2DTap = useCallback(() => {
  mainManager.onLive2DTap();
}, []);

// 双指长按拖动状态
const dragStartPositionRef = useRef<{ x: number; y: number } | null>(null);
// 使用独立 ref 跟踪当前位置，不依赖 React 状态（因为 setPosition 不会触发 React 更新）
const currentModelPositionRef = useRef({ x: 0, y: 0 });
const [isDraggingModel, setIsDraggingModel] = useState(false);

// 双指缩放状态
const startScaleRef = useRef<number>(0.8);
const currentScaleRef = useRef<number>(0.8);
const [isScalingModel, setIsScalingModel] = useState(false);

// 拖动手势
const dragGesture = useMemo(() => {
  let screenWidth = 1;
  let screenHeight = 1;
  return Gesture.Pan()
    .minPointers(2)
    .activateAfterLongPress(500)
    .runOnJS(true)
    .onStart(() => {
      const { width, height } = Dimensions.get('window');
      screenWidth = width;
      screenHeight = height;
      // 使用持久化的位置 ref，而不是 React 状态
      const pos = { ...currentModelPositionRef.current };
      if (Math.abs(pos.x) > POSITION_LIMIT || Math.abs(pos.y) > POSITION_LIMIT) {
        live2d.setModelPosition(0, 0);
        currentModelPositionRef.current = { x: 0, y: 0 };
        pos.x = 0;
        pos.y = 0;
      }
      dragStartPositionRef.current = pos;
      setIsDraggingModel(true);
    })
    .onUpdate((e) => {
      const start = dragStartPositionRef.current;
      if (!start) return;
      // 大幅降低灵敏度：手指移动整个屏幕距离，模型仅移动 0.005 个逻辑单位
      // 乘数越小灵敏度越低，0.005 = 低灵敏度（需要大幅度拖动才能移动模型）
      const sensitivity = 0.005;
      const newX = clampPos(start.x + (e.translationX / screenWidth) * sensitivity);
      const newY = clampPos(start.y - (e.translationY / screenHeight) * sensitivity);
      // 更新当前位置 ref，供下次拖动使用
      currentModelPositionRef.current = { x: newX, y: newY };
      live2d.setModelPosition(newX, newY);
    })
    .onFinalize(() => {
      // 拖动结束时，保存最终位置到 ref（从 dragStartPositionRef 计算最终位置）
      // 这样下次拖动时可以从上次结束的位置开始
      dragStartPositionRef.current = null;
      setIsDraggingModel(false);
    });
// eslint-disable-next-line react-hooks/exhaustive-deps
}, []);

// 双指缩放手势（捏合/张开）
// 注意：Pinch 手势本身就需要双指，无需 minPointers
const pinchGesture = useMemo(() => {
  return Gesture.Pinch()
    .runOnJS(true)
    .onStart(() => {
      // 记录开始时的缩放值
      startScaleRef.current = currentScaleRef.current;
      setIsScalingModel(true);
    })
    .onUpdate((e) => {
      // 降低缩放灵敏度：缩放因子变化更平缓
      const scaleSensitivity = 0.5;
      const newScale = clampScale(startScaleRef.current * (1 + (e.scale - 1) * scaleSensitivity));
      currentScaleRef.current = newScale;
      live2d.setModelScale(newScale);
    })
    .onFinalize(() => {
      setIsScalingModel(false);
```

```
    });
// eslint-disable-next-line react-hooks/exhaustive-deps
}, []);

// 组合手势：双指拖动 + 缩放同时识别
const live2dGesture = useMemo(() => {
  return Gesture.Simultaneous(dragGesture, pinchGesture);
}, [dragGesture, pinchGesture]);

// 工具栏事件处理（与 Web 版本一致）
const handleToolbarSettingsChange = useCallback((id: Live2DSettingsToggleId, next: boolean) => {
  setToolbarSettings((prev) => ({ ...prev, [id]: next }));
  if (id === 'darkMode') {
    Appearance.setColorScheme(next ? 'dark' : 'light');
  }
}, []);

const handleToolbarAgentChange = useCallback((id: Live2DAgentToggleId, next: boolean) => {
  onAgentChange(id, next);
}, [onAgentChange]);

const handleToggleMic = useCallback(async (next: boolean) => {
  setToolbarMicEnabled(next);
  if (next) {
    setVoicePrepareStatus('preparing');
    try {
      await mainManager.startRecording();
      // 清除之前的错误提示（如果有）
      setSwitchError(null);
      if (switchErrorTimerRef.current) {
        clearTimeout(switchErrorTimerRef.current);
        switchErrorTimerRef.current = null;
      }
      setVoicePrepareStatus('ready');
      setTimeout(() => setVoicePrepareStatus(null), 800);
    } catch {
      setVoicePrepareStatus(null);
      setToolbarMicEnabled(false);
      // 显示与角色切换失败一致的底部错误横幅
      setSwitchError('语音系统准备失败');
      if (switchErrorTimerRef.current) clearTimeout(switchErrorTimerRef.current);
      switchErrorTimerRef.current = setTimeout(() => setSwitchError(null), 3000);
    }
  } else {
    mainManager.stopRecording();
    // 语音会话停止后，重置 text session 状态
    // 这样下次发送文本消息时会重新发送 start_session
    setIsTextSessionActive(false);
  }
}, [mainManager]);

const handleToggleScreen = useCallback(async (next: boolean) => {
  if (!next) {
    // 停止摄像头
    cameraStream.stopStreaming();
    statusToastRef.current?.show('已停止摄像头', 2000);
    return;
  }

  // 先检查权限
  const hasPermission = await cameraStream.checkAndRequestPermission();
  if (!hasPermission) return;

  // 显示选择对话框
  Alert.alert(
    '选择摄像头',
    '请选择要使用的摄像头',
    [
      {
        text: '前置摄像头',
        onPress: () => {
          cameraStream.startStreaming('front');
          statusToastRef.current?.show('前置摄像头已开启', 2000);
        },
      },
      {
        text: '后置摄像头',
        onPress: () => {
          cameraStream.startStreaming('back');
          statusToastRef.current?.show('后置摄像头已开启', 2000);
        },
      },
      {
        text: '取消',
```

```
      style: 'cancel',
    },
    ]
  );
}, [cameraStream.startStreaming, cameraStream.stopStreaming, cameraStream.checkAndRequestPermission]);

const handleGoodbye = useCallback(() => {
  // 如果麦克风正在录音，先停止
  if (toolbarMicEnabled) {
    mainManager.stopRecording();
    setToolbarMicEnabled(false);
    // 语音会话停止后，重置 text session 状态
    setIsTextSessionActive(false);
  }
  // 如果摄像头流正在运行，停止
  if (cameraStream.isStreaming) {
    cameraStream.stopStreaming();
  }
  setToolbarGoodbyeMode(true);
  setToolbarOpenPanel(null);
}, [mainManager, toolbarMicEnabled, cameraStream.isStreaming, cameraStream.stopStreaming]);

const handleReturn = useCallback(() => {
  setToolbarGoodbyeMode(false);
}, []);

const handleSettingsMenuClick = useCallback((id: string) => {
  if (id === 'characterManage') {
    const loadCharacters = async () => {
      try {
        setCharacterLoading(true);
        const apiBase = buildHttpBaseURL(config);
        const client = createCharactersApiClient(apiBase, config.p2p?.token);
        const data: CharactersData = await client.getCharacters();

        const names = Object.keys(data.猫娘 || {});
        if (names.length === 0) {
          Alert.alert(t('characterManager.title'), t('characterManager.empty'));
          return;
        }

        setCharacterList(names);
        setCurrentCatgirl(data.当前猫娘 || null);
        setToolbarOpenPanel(null);
        setCharacterModalVisible(true);
      } catch (err: any) {
        Alert.alert(t('characterManager.title'), err.message || t('connection.errors.networkError'));
      } finally {
        setCharacterLoading(false);
      }
    };
    loadCharacters();
    return;
  }
  if (id === 'reload') {
    setToolbarOpenPanel(null);

    const doReload = async () => {
      try {
        // 1. 如果正在录音，先停止
        if (toolbarMicEnabled) {
          mainManager.stopRecording();
          setToolbarMicEnabled(false);
          setIsTextSessionActive(false);
        }

        // 2. 清空聊天记录
        chat.clearMessages();

        // 3. 重连 WebSocket
        audio.reconnect();

        // 4. 重新加载角色 + Live2D 模型
        const apiBase = buildHttpBaseURL(config);
        const client = createCharactersApiClient(apiBase, config.p2p?.token);
        const res = await client.getCurrentCatgirl();
        if (res.current_catgirl) {
          setCurrentCatgirl(res.current_catgirl);
          if (config.characterName !== res.current_catgirl) {
            await setConfig({ ...config, characterName: res.current_catgirl });
          }
          await syncLive2dModel(res.current_catgirl);
        }
```

```
      statusToastRef.current?.show('重新加载完成', 2000);
    } catch (e: any) {
      console.warn(' 重新加载失败:', e);
      statusToastRef.current?.show('重新加载失败: ' + (e.message || '未知错误'), 3000);
    }
  };
  doReload();
  return;
 }
 if (id === 'connectionHelp') {
  Alert.alert(
    '连接帮助',
    '如果手机无法连接电脑，可能是网络环境限制（Symmetric NAT）。\n\n' +
    '解决方法：在电脑端安装内网穿透工具：\n' +
    '• cpolar（推荐）: cpolar.com\n' +
    '• natapp: natapp.cn\n\n' +
    '安装后将本地端口 48920 映射到公网，用获取的公网地址重新扫码连接。'
  );
  return;
 }

 Alert.alert(t('common.alert'), `即将打开: ${id}`);
}, [config, toolbarMicEnabled, mainManager, chat, audio, syncLive2dModel]);

const handleSwitchCharacter = useCallback(async (name: string) => {
 // 检查是否在语音模式
 if (toolbarMicEnabled) {
  setVoiceBlockModalVisible(true);
  return;
 }

 try {
  setCharacterLoading(true);
  const apiBase = buildHttpBaseURL(config);
  const client = createCharactersApiClient(apiBase, config.p2p?.token);
  const res = await client.setCurrentCatgirl(name);

  if (res.success) {
    setCharacterModalVisible(false);
    // 立即置为切换中，屏蔽切换期间的 WebSocket 错误（不等服务端广播）
    isSwitchingCharacterRef.current = true;
    // UI 更新由服务端广播的 catgirl_switched 消息统一驱动
    // 超时保护：15 秒内若未收到 onConnectionChange(true)，自动解除所有切换状态
    if (characterLoadingTimerRef.current) clearTimeout(characterLoadingTimerRef.current);
    characterLoadingTimerRef.current = setTimeout(() => {
      setCharacterLoading(false);
      setIsChatForceCollapsed(false);
      isSwitchingCharacterRef.current = false;
      characterLoadingTimerRef.current = null;
      setSwitchError('连接超时，角色切换失败');
      if (switchErrorTimerRef.current) clearTimeout(switchErrorTimerRef.current);
      switchErrorTimerRef.current = setTimeout(() => setSwitchError(null), 3000);
    }, 15000);
  } else {
    setCharacterLoading(false);
    Alert.alert(t('main.character.switchError'), res.error || t('common.error'));
  }
 } catch (err: any) {
  setCharacterLoading(false);
  Alert.alert(t('main.character.switchError'), err.message || t('connection.errors.networkError'));
 }
}, [config, toolbarMicEnabled]);

// 确保 text session 已启动（与 Web 端一致的 Legacy 协议）
const ensureTextSession = useCallback(async (): Promise<boolean> => {
 // 如果已经有活跃的 text session，直接返回
 if (isTextSessionActive) {
  return true;
 }

 if (!audio.isConnected) {
  return false;
 }

 // 如果当前正在录音（语音模式），先停止录音并等待服务端清理旧 session，
 // 避免 start_session(text) 与正在启动/活跃的 audio session 产生竞态
 if (audio.isRecording) {
  console.log(' 检测到正在录音，先停止录音再切换到文本模式');
  await audio.toggleRecording();
  // 给服务端一点时间完成 end_session 清理
  await new Promise(r => setTimeout(r, 500));
  // 500ms 等待后重检：若用户在此期间重新开始录音，则放弃本次文本发送
  if (audio.isRecording) {
    console.log(' 等待期间用户重新开始录音，放弃 text session');
```

```
    return false;
  }
}

// 如果已经有一个正在进行的 session 请求，复用该 Promise
// 这样并发调用会共享同一个 Promise，避免 resolver 被覆盖导致早期 Promise 永不 resolve
if (pendingSessionPromiseRef.current) {
  return pendingSessionPromiseRef.current;
}

const promise = new Promise<boolean>((resolve) => {
  // 清除任何现有的超时
  if (sessionTimeoutRef.current) {
    clearTimeout(sessionTimeoutRef.current);
    sessionTimeoutRef.current = null;
  }

  // 设置 resolver
  sessionStartedResolverRef.current = resolve;

  // 发送 start_session（Legacy 协议）
  console.log(' 发送 start_session(input_type: text, audio_format: PCM_48000HZ_MONO_16BIT)');
  audio.sendMessage({
    action: 'start_session',
    input_type: 'text',
    audio_format: 'PCM_48000HZ_MONO_16BIT',
    new_session: false,
  });

  // 15 秒超时
  sessionTimeoutRef.current = setTimeout(() => {
    if (sessionStartedResolverRef.current === resolve) {
      sessionStartedResolverRef.current = null;
      pendingSessionPromiseRef.current = null;
      console.log(' start_session 超时');
      resolve(false);
    }
    sessionTimeoutRef.current = null;
  }, 15000);
});

pendingSessionPromiseRef.current = promise;
return promise;
}, [isTextSessionActive, audio.isConnected, audio.isRecording, audio.sendMessage, audio.toggleRecording]);

// 图片消息服务
const imageMessageService = useMemo(() => new ImageMessageService(), []);

// 处理用户发送消息（文本 + 可选图片）
// 使用 stream_data action 和 clientMessageId 与 N.E.K.O 协议一致
const handleSendMessage = useCallback(async (text: string, images?: string[]) => {
  if (!audio.isConnected) {
    Alert.alert(t('common.alert'), t('connection.status.disconnected'));
    return;
  }

  // 确保 text session 已启动（与 Web 端一致）
  const sessionOk = await ensureTextSession();
  if (!sessionOk) {
    Alert.alert(t('common.alert'), t('connection.status.reconnecting'));
    return;
  }

  // 合并图片来源：
  // 1. imagePicker.images - 相册选择的图片（expo-image-picker 返回的 base64）
  // 2. images 参数 - ChatContainer 传入的 pendingScreenshots（相机拍照的 base64）
  const imagesToSend: string[] = [];

  // 处理相册选择的图片（直接使用 base64，不再重新压缩）
  if (imagePicker.images.length > 0) {
    console.log(' 处理相册图片...');
    for (const img of imagePicker.images) {
      // expo-image-picker 返回的 base64 是纯 base64，需要添加 data URI 前缀
      const mimeType = img.mimeType || 'image/jpeg';
      const base64WithPrefix = img.base64.startsWith('data:')
        ? img.base64
        : `data:${mimeType};base64,${img.base64}`;
      imagesToSend.push(base64WithPrefix);
    }
    console.log(` 相册图片处理完成：${imagePicker.images.length} 张`);
  }

  // 处理传入的图片（相机拍照的，已经是 base64 格式）
  if (images && images.length > 0) {
```

```
    console.log(' 处理相机图片...');
    for (const img of images) {
      // 传入的图片可能已经有前缀，也可能没有
      const base64WithPrefix = img.startsWith('data:')
        ? img
        : `${img}`;
      imagesToSend.push(base64WithPrefix);
    }
    console.log(` 相机图片处理完成：${images.length} 张`);
  }

  // 发送图片
  if (imagesToSend.length > 0) {
    for (const imgBase64 of imagesToSend) {
      messageCounterRef.current += 1;
      const clientMessageId = generateMessageId(messageCounterRef.current);
      sentClientMessageIds.current.set(clientMessageId, Date.now());

      audio.sendMessage({
        action: 'stream_data',
        data: imgBase64,
        input_type: 'camera', // 与 N.E.K.O 主项目兼容
        clientMessageId,
      });
    }
    chat.addMessage(` [已发送${imagesToSend.length}张照片]`, 'user');
  }

  // 再发送文本
  if (text.trim()) {
    messageCounterRef.current += 1;
    const clientMessageId = generateMessageId(messageCounterRef.current);
    sentClientMessageIds.current.set(clientMessageId, Date.now());

    // 添加用户消息到 UI
    chat.addMessage(text, 'user');

    // 通过 WS 发送到后端（使用 stream_data action，与 N.E.K.O 协议一致）
    audio.sendMessage({
      action: 'stream_data',
      data: text.trim(),
      input_type: 'text',
      clientMessageId,
    });

    console.log(' 发送文本消息:', text.substring(0, 50));
  }

  // 清除已选图片
  imagePicker.clearImages();
  setCameraPendingImages([]);
}, [audio.isConnected, audio.sendMessage, chat.addMessage, ensureTextSession, imagePicker, imageMessageService, setCameraPendingImages]);

// 检测屏幕尺寸变化
useEffect(() => {
  const updateIsMobile = () => {
    const { width, height } = Dimensions.get('window');
    setIsMobile(width <= 768);
    setScreenHeight(height);
  };

  updateIsMobile();
  const subscription = Dimensions.addEventListener('change', updateIsMobile);

  return () => {
    subscription?.remove();
  };
}, []);

// 清理切换相关 timer，防止组件卸载后 setState
useEffect(() => {
  return () => {
    if (switchedNameTimerRef.current) clearTimeout(switchedNameTimerRef.current);
    if (switchErrorTimerRef.current) clearTimeout(switchErrorTimerRef.current);
    if (characterLoadingTimerRef.current) clearTimeout(characterLoadingTimerRef.current);
  };
}, []);

// 显示 Agent 状态（调试用）
useEffect(() => {
  console.log(' Agent 状态:', agent.statusText, {
    master: agent.master,
    keyboard: agent.keyboard,
    mcp: agent.mcp,
```

```
    userPlugin: agent.userPlugin,
  });
}, [agent]);

return (
  <View style={styles.container}>
    {/* 状态提示 Toast */}
    <StatusToast ref={statusToastRef} />

    {/* Live2D 舞台区域 */}
    {/* Android 端：GestureDetector 包裹整个容器，同时支持单指注视（原生处理）和双指手势 */}
    {/* iOS 端：直接渲染容器，暂不支持双指手势 */}
    {Platform.OS === 'android' ? (
      <GestureDetector gesture={live2dGesture}>
        <View style={styles.live2dContainer}>
          {/* 页面获得焦点时渲染 Live2D */}
          {isPageFocused && (
            <ReactNativeLive2dView
              style={styles.live2dView}
              {...live2d.live2dPropsForLipSync}
              onTap={handleLive2DTap}
            />
          )}

          {/* 失去焦点时的显示 */}
          {!isPageFocused && (
            <View style={styles.pausedContainer}>
              <Text style={styles.pausedText}>
                {live2d.live2dProps.modelPath ? 'Live2D 已暂停' : '页面未激活'}
              </Text>
            </View>
          )}

          {(isDraggingModel || isScalingModel) && (
            <View style={styles.dragIndicator} pointerEvents="none">
              <Text style={styles.dragIndicatorText}>
                {isDraggingModel && isScalingModel ? '拖动/缩放中' : isDraggingModel ? '拖动中' : '缩放中'}
              </Text>
            </View>
          )}
        </View>
      </GestureDetector>
    ) : (
      <View style={styles.live2dContainer}>
        {/* iOS 端暂不支持双指手势 */}
        {isPageFocused && (
          <ReactNativeLive2dView
            style={styles.live2dView}
            {...live2d.live2dPropsForLipSync}
            onTap={handleLive2DTap}
          />
        )}

        {!isPageFocused && (
          <View style={styles.pausedContainer}>
            <Text style={styles.pausedText}>
              {live2d.live2dProps.modelPath ? 'Live2D 已暂停' : '页面未激活'}
            </Text>
          </View>
        )}
      </View>
    )}

    {/*
      【跨平台组件】Live2DRightToolbar 右侧工具栏

      策略更新（2026-01-11）：
      - 已实现 RN 原生版本（Live2DRightToolbar.native.tsx）
      - 使用共享的类型和业务逻辑（types.ts + hooks.ts）
      - Metro Bundler 自动根据平台选择：
        * Web: Live2DRightToolbar.tsx（HTML/CSS 完整版）
        * Android/iOS: Live2DRightToolbar.native.tsx（Modal 简化版）
      - 详见：docs/strategy/cross-platform-components.md

// ------------------------------------------------
// BACK 30 PAGES
// ------------------------------------------------


  /**
   * 设置响应式监听（仅在Web环境）
   */
  setupResizeListener() {
    // 检查是否在Web环境且window对象可用
```

```
    if (typeof window === 'undefined' || !window.addEventListener) {
        console.log(' 非Web环境或window不可用，跳过resize监听器设置');
        return () => {}; // 返回空的清理函数
    }

    const handleResize = () => {
        const deviceAdapter = Live2DConfigManager.createDeviceAdapter();
        const newIsMobile = deviceAdapter.isMobile();
        const newScreenSize = deviceAdapter.getScreenSize();
        const newPerformanceLevel = deviceAdapter.getPerformanceLevel();

        this.updateConfig({
            deviceInfo: {
                ...this.config.deviceInfo,
                isMobile: newIsMobile,
                screenSize: newScreenSize,
                performanceLevel: newPerformanceLevel
            }
        });
    };

    window.addEventListener('resize', handleResize);
    console.log(' Web环境resize监听器已设置');

    // 返回清理函数
    return () => {
        if (typeof window !== 'undefined' && window.removeEventListener) {
            window.removeEventListener('resize', handleResize);
        }
    };
}

/**
 * 更新配置
 */
updateConfig(newConfig) {
    if (!this.config) {
        console.warn('配置管理器尚未初始化');
        return;
    }

    const oldConfig = { ...this.config };
    this.config = {
        ...this.config,
        ...newConfig
    };

    console.log(' Live2D 配置已更新:', {
        old: oldConfig,
        new: this.config,
        changes: newConfig
    });

    // 通知所有监听器
    this.notifyListeners(this.config, oldConfig);
}

/**
 * 获取当前配置
 */
getConfig() {
    return this.config;
}

/**
 * 添加配置变化监听器
 */
addListener(listener) {
    this.listeners.add(listener);

    // 返回移除监听器的函数
    return () => {
        this.listeners.delete(listener);
    };
}

/**
 * 通知所有监听器
 */
notifyListeners(newConfig, oldConfig) {
    this.listeners.forEach(listener => {
        try {
            listener(newConfig, oldConfig);
        } catch (error) {
```

```
            console.error('配置监听器执行失败:', error);
        }
    });
  }

  /**
   * 获取适配后的模型设置参数
   */
  getModelSettings() {
    if (!this.config) return null;

    const { deviceInfo, debugMode } = this.config;
    const { isMobile } = deviceInfo;

    let scale, position;

    if (isMobile) {
      scale = this.config.mobileScale;
      position = this.config.mobilePosition;
    } else {
      scale = this.config.desktopScale;
      position = this.config.desktopPosition;
    }

    // 调试模式覆盖
    if (debugMode) {
      const debugScale = isMobile ? this.config.debugScale.mobile : this.config.debugScale.desktop;
      const debugPosition = isMobile ? this.config.debugPosition.mobile : this.config.debugPosition.desktop;

      return {
        scale: debugScale,
        position: debugPosition,
        isDebugMode: true,
        isMobile
      };
    }

    return {
      scale,
      position,
      isDebugMode: false,
      isMobile
    };
  }

  /**
   * 生成WebView通信用的简化配置
   */
  getWebViewConfig() {
    if (!this.config) return null;

    return {
      ...this.config,
      // 添加一些WebView特定的字段
      timestamp: Date.now(),
      platform: this.platform
    };
  }

  /**
   * 销毁配置管理器
   */
  destroy() {
    this.listeners.clear();
    this.config = null;
    console.log(' Live2D 配置管理器已销毁');
  }
}

// 导出单例实例
export const live2dConfigManager = new Live2DConfigManager();

// 导出为全局变量（用于HTML环境）
if (typeof window !== 'undefined') {
  window.Live2DConfigManager = Live2DConfigManager;
  window.live2dConfigManager = live2dConfigManager;
}

// ----------------------------------------

import { AudioService } from '@/services/AudioService';
import { Live2DService } from '@/services/Live2DService';

/**
```

```
 * 情感类型枚举
 */
export enum EmotionType {
  NEUTRAL = 'neutral',
  HAPPY = 'happy',
  SAD = 'sad',
  SURPRISED = 'surprised',
  ANGRY = 'angry',
}

/**
 * 情感到表情的映射
 */
const EMOTION_TO_EXPRESSION_MAP: Record<EmotionType, string> = {
  [EmotionType.NEUTRAL]: 'exp_exp_01',
  [EmotionType.HAPPY]: 'exp_exp_02',
  [EmotionType.SAD]: 'exp_exp_03',
  [EmotionType.SURPRISED]: 'exp_exp_04',
  [EmotionType.ANGRY]: 'exp_exp_05',
};

/**
 * 情感到动作的映射
 */
const EMOTION_TO_MOTION_MAP: Record<EmotionType, string> = {
  [EmotionType.NEUTRAL]: 'Idle',
  [EmotionType.HAPPY]: 'happy',
  [EmotionType.SAD]: 'sad',
  [EmotionType.SURPRISED]: 'surprised',
  [EmotionType.ANGRY]: 'sad', // 可以自定义
};

/**
 * MainManager - 主管理器（协调层）
 *
 * 职责：
 * - 协调 Audio 和 Live2D 服务之间的交互
 * - 根据音频事件触发 Live2D 动作/表情
 * - 管理全局状态和配置
 * - 提供统一的业务逻辑接口
 */
class MainManager {
  private audioService: AudioService | null = null;
  private live2dService: Live2DService | null = null;
  private isInitialized: boolean = false;
  private currentEmotion: EmotionType = EmotionType.NEUTRAL;

  /**
   * 初始化管理器
   */
  init() {
    if (this.isInitialized) {
      console.warn(' MainManager 已经初始化过了');
      return;
    }

    console.log(' MainManager 初始化中...');
    this.isInitialized = true;
    console.log(' MainManager 初始化完成');
  }

  /**
   * 注册 Audio 服务
   */
  registerAudioService(audioService: AudioService) {
    console.log(' 注册 AudioService');
    this.audioService = audioService;
  }

  /**
   * 注册 Live2D 服务
   */
  registerLive2DService(live2dService: Live2DService) {
    console.log(' 注册 Live2DService');
    this.live2dService = live2dService;
  }

  /**
   * 处理 Gemini 响应（AI 开始说话）
   */
  onGeminiResponse(isNewMessage: boolean) {
    console.log(' 处理 Gemini 响应, isNewMessage:', isNewMessage);

    if (isNewMessage) {
```

```
    // 新消息开始，清空音频队列
    this.audioService?.clearAudioQueue();

    // Live2D 做出反应（开心的动作）
    this.applyEmotion(EmotionType.HAPPY);
  }
}

/**
 * 处理用户语音活动（用户说话，打断 AI）
 */
onUserSpeechDetected() {
  console.log(' 检测到用户语音活动');

  // 处理音频打断
  this.audioService?.handleUserSpeechDetection();

  // Live2D 做出反应（惊讶的动作）
  this.applyEmotion(EmotionType.SURPRISED);
}

/**
 * 处理回合结束
 */
onTurnEnd(fullText?: string) {
  console.log(' 回合结束');

  // TODO: 在这里可以进行情感分析
  // const emotion = await analyzeEmotion(fullText);
  // this.applyEmotion(emotion);

  // 暂时使用中性表情
  this.applyEmotion(EmotionType.NEUTRAL);
}

/**
 * 应用情感（同时改变 Live2D 的动作和表情）
 */
applyEmotion(emotion: EmotionType) {
  console.log(' 应用情感:', emotion);
  this.currentEmotion = emotion;

  if (!this.live2dService) {
    console.warn(' Live2DService 未注册，无法应用情感');
    return;
  }

  // 获取对应的表情和动作
  const expression = EMOTION_TO_EXPRESSION_MAP[emotion] || 'exp_exp_01';
  const motion = EMOTION_TO_MOTION_MAP[emotion] || 'Idle';

  // 应用到 Live2D
  this.live2dService.setExpression(expression);
  this.live2dService.playMotion(motion);
}

/**
 * 播放特定动作
 */
playMotion(motion: string) {
  console.log(' 播放动作:', motion);
  this.live2dService?.playMotion(motion);
}

/**
 * 设置表情
 */
setExpression(expression: string) {
  console.log(' 设置表情:', expression);
  this.live2dService?.setExpression(expression);
}

/**
 * 开始录音
 */
async startRecording() {
  console.log(' 开始录音');
  await this.audioService?.startRecording();

  // Live2D 做出倾听的姿态
  this.live2dService?.playMotion('Idle');
}

/**
```

```
  * 停止录音
  */
  async stopRecording() {
    console.log(' 停止录音');
    await this.audioService?.stopRecording();
  }

  /**
  * 切换录音状态
  */
  async toggleRecording() {
    const isRecording = this.audioService?.getIsRecording();
    if (isRecording) {
      await this.stopRecording();
    } else {
      await this.startRecording();
    }
  }

  /**
  * 处理用户点击 Live2D 模型
  */
  onLive2DTap() {
    console.log(' 用户点击了 Live2D 模型');

    // 播放一个随机动作
    const motions = ['happy', 'surprised', 'neutral'];
    const randomMotion = motions[Math.floor(Math.random() * motions.length)];
    this.live2dService?.playMotion(randomMotion);
  }

  /**
  * 获取当前情感
  */
  getCurrentEmotion(): EmotionType {
    return this.currentEmotion;
  }

  /**
  * 获取 Audio 服务
  */
  getAudioService(): AudioService | null {
    return this.audioService;
  }

  /**
  * 获取 Live2D 服务
  */
  getLive2DService(): Live2DService | null {
    return this.live2dService;
  }

  /**
  * 是否已初始化
  */
  isReady(): boolean {
    return this.isInitialized;
  }

  /**
  * 销毁管理器
  */
  destroy() {
    console.log(' MainManager 销毁中...');
    this.audioService = null;
    this.live2dService = null;
    this.isInitialized = false;
    this.currentEmotion = EmotionType.NEUTRAL;
    console.log(' MainManager 已销毁');
  }
}

/**
* 全局单例实例
*/
export const mainManager = new MainManager();

// ----------------------------------------

// RN_AudioProcessor.ts - React Native Android 音频处理器
import { Buffer } from 'buffer';

interface AudioProcessorOptions {
  originalSampleRate?: number;
```

```
  targetSampleRate?: number;
  bufferSize?: number;
  onAudioFrame?: (pcmData: Int16Array) => void;
}

export default class AudioProcessor {
  private resampleRatio: number;
  private bufferSize: number;
  private tempBuffer: number[];
  private onAudioFrame?: (pcmData: Int16Array) => void;
  private isDestroyed: boolean = false;

  constructor(options: AudioProcessorOptions = {}) {
    const {
      originalSampleRate = 48000,
      targetSampleRate = 16000,
      bufferSize = 512,
      onAudioFrame
    } = options;

    this.resampleRatio = targetSampleRate / originalSampleRate;
    this.bufferSize = bufferSize;
    this.tempBuffer = [];
    this.onAudioFrame = onAudioFrame;

    console.log(`RN_AudioProcessor 初始化: 原始采样率=${originalSampleRate}Hz, 目标采样率=${targetSampleRate}Hz, 缓冲区=${bufferSize}帧`);
  }

  // 处理 base64 PCM 输入（Android 优化版）
  pushAudioChunk(base64Data: string): void {
    if (this.isDestroyed || !base64Data || !this.onAudioFrame) return;

    try {
      const pcm = this.base64ToPCM(base64Data);
      if (!pcm || pcm.length === 0) return;

      // 转成 Float32 [-1,1]，Android 优化：避免过度归一化
      const float32 = Float32Array.from(pcm, x => Math.max(-1, Math.min(1, x / 32768)));
      this.tempBuffer = this.tempBuffer.concat(Array.from(float32));

      const requiredSamples = Math.ceil(this.bufferSize / this.resampleRatio);

      // Android 优化：限制处理循环次数，避免阻塞主线程
      let processCount = 0;
      const maxProcessCount = 10; // 限制单次处理的最大循环次数

      while (this.tempBuffer.length >= requiredSamples && processCount < maxProcessCount) {
        const samplesToProcess = this.tempBuffer.slice(0, requiredSamples);
        this.tempBuffer = this.tempBuffer.slice(requiredSamples);

        if (samplesToProcess.length === 0) break;

        // 重采样并转成 Int16 PCM
        const resampledData = this.resampleAudio(samplesToProcess);
        if (!resampledData || resampledData.length === 0) {
          processCount++;
          continue;
        }

        const pcmData = new Int16Array(resampledData.length);
        for (let i = 0; i < resampledData.length; i++) {
          // Android 优化：确保数值在有效范围内
          const clampedValue = Math.max(-1, Math.min(1, resampledData[i]));
          pcmData[i] = Math.round(clampedValue * 0x7FFF);
        }

        this.onAudioFrame(pcmData);
        processCount++;
      }

      // Android 优化：如果缓冲区过大，清理部分数据避免内存泄漏
      if (this.tempBuffer.length > requiredSamples * 10) {
        this.tempBuffer = this.tempBuffer.slice(-requiredSamples * 5);
      }
    } catch (error) {
      console.error('RN_AudioProcessor pushAudioChunk 错误:', error);
    }
  }

  // base64 → Int16Array（Android 优化版）
  private base64ToPCM(base64: string): Int16Array {
    try {
      if (!base64 || base64.length % 4 !== 0) return new Int16Array(0);
```

```
        const buffer = Buffer.from(base64, 'base64');
        if (!buffer || buffer.length === 0 || buffer.length % 2 !== 0) return new Int16Array(0);

        // Android 优化：使用更高效的 ArrayBuffer 创建方式
        const result = new Int16Array(buffer.buffer, buffer.byteOffset, buffer.length / 2);

        // Android 优化：验证数据完整性
        if (result.length === 0) return new Int16Array(0);

        return result;
      } catch (error) {
        console.error('RN_AudioProcessor base64ToPCM 错误:', error);
        return new Int16Array(0);
      }
    }

    // 线性插值重采样（Android 优化版）
    private resampleAudio(audioData: number[]): Float32Array {
      try {
        if (!audioData || audioData.length === 0 || this.resampleRatio <= 0) {
          return new Float32Array(0);
        }

        const inputLength = audioData.length;
        const outputLength = Math.floor(inputLength * this.resampleRatio);
        if (outputLength <= 0) return new Float32Array(0);

        const result = new Float32Array(outputLength);

        // Android 优化：使用更高效的循环和数值处理
        for (let i = 0; i < outputLength; i++) {
          const position = i / this.resampleRatio;
          const index = Math.floor(position);
          const fraction = position - index;

          if (index + 1 < inputLength) {
            // 线性插值，Android 优化：减少函数调用
            const val1 = audioData[index];
            const val2 = audioData[index + 1];
            result[i] = val1 + (val2 - val1) * fraction;
          } else if (index < inputLength) {
            result[i] = audioData[index];
          } else {
            result[i] = 0;
          }
        }

        return result;
      } catch (error) {
        console.error('RN_AudioProcessor resampleAudio 错误:', error);
        return new Float32Array(0);
      }
    }

    // 清空缓冲区（Android 优化版）
    clearBuffer(): void {
      this.tempBuffer = [];
    }

    // 设置音频帧回调（Android 优化版）
    setAudioFrameCallback(callback?: (pcmData: Int16Array) => void): void {
      this.onAudioFrame = callback;
    }

    // 获取缓冲区状态（Android 调试用）
    getBufferStatus(): { length: number; bufferSize: number; isDestroyed: boolean } {
      return {
        length: this.tempBuffer.length,
        bufferSize: this.bufferSize,
        isDestroyed: this.isDestroyed
      };
    }

    // 销毁处理器（Android 优化版）
    destroy(): void {
      this.isDestroyed = true;
      this.tempBuffer = [];
      this.onAudioFrame = undefined;
    }
  }

  // ----------------------------------------

  /**
```

```
 * 音频诊断工具
 * 用于调试 Android 录音权限和配置问题
 */

import { Platform, PermissionsAndroid } from 'react-native';
import PCMStream from 'react-native-pcm-stream';

export interface AudioDiagnosticsResult {
  platform: string;
  permissionGranted: boolean | null;
  canInitializeAudioRecord: boolean;
  supportedSampleRates: number[];
  errorMessage?: string;
  nativeError?: string;
}

/**
 * 运行音频诊断
 */
export async function runAudioDiagnostics(): Promise<AudioDiagnosticsResult> {
  const result: AudioDiagnosticsResult = {
    platform: Platform.OS,
    permissionGranted: null,
    canInitializeAudioRecord: false,
    supportedSampleRates: [],
  };

  console.log(' 开始音频诊断...');

  // 1. 检查平台
  if (Platform.OS !== 'android') {
    result.errorMessage = '当前平台不是 Android';
    console.log(' 当前平台不是 Android，跳过诊断');
    return result;
  }

  // 2. 检查权限
  try {
    const permission = PermissionsAndroid.PERMISSIONS.RECORD_AUDIO;
    const checkResult = await PermissionsAndroid.check(permission);
    result.permissionGranted = checkResult;
    console.log(` 麦克风权限状态: ${checkResult ? '已授予 ' : '未授予 '}`);

    if (!checkResult) {
      // 尝试请求权限
      console.log(' 尝试请求麦克风权限...');
      const requestResult = await PermissionsAndroid.request(permission);
      const granted = requestResult === PermissionsAndroid.RESULTS.GRANTED;
      result.permissionGranted = granted;
      console.log(`${granted ? ' 权限已授予' : ' 权限被拒绝'}`);

      if (!granted) {
        result.errorMessage = '用户拒绝了麦克风权限';
        return result;
      }
    }
  } catch (error: any) {
    result.errorMessage = `权限检查失败: ${error.message}`;
    console.error(' 权限检查异常:', error);
    return result;
  }

  // 3. 测试不同采样率
  const testSampleRates = [8000, 16000, 22050, 44100, 48000];
  console.log(' 测试支持的采样率...');

  // 监听原生错误
  let lastNativeError: string | null = null;
  const errorListener = PCMStream.addListener('onError', (event: any) => {
    lastNativeError = event?.message || 'Unknown error';
    console.error(' 原生错误:', lastNativeError);
  });

  for (const sampleRate of testSampleRates) {
    try {
      lastNativeError = null;
      console.log(` 测试 ${sampleRate}Hz...`);

      // 尝试初始化录音（会立即停止）
      PCMStream.startRecording(sampleRate, 1024, sampleRate);

      // 短暂等待看是否有错误
      await new Promise(resolve => setTimeout(resolve, 100));
```

```
      if (!lastNativeError) {
        result.supportedSampleRates.push(sampleRate);
        console.log(` ${sampleRate}Hz 支持`);
      } else {
        console.log(` ${sampleRate}Hz 不支持: ${lastNativeError}`);
      }

      // 停止录音
      try {
        PCMStream.stopRecording();
      } catch (_e) {
        // 忽略停止错误
      }

      // 等待资源释放
      await new Promise(resolve => setTimeout(resolve, 200));
    } catch (error: any) {
      console.log(` ${sampleRate}Hz 测试失败:`, error.message);
      if (lastNativeError) {
        result.nativeError = lastNativeError;
      }
    }
  }

  errorListener.remove();

  // 4. 总结
  result.canInitializeAudioRecord = result.supportedSampleRates.length > 0;

  console.log('\n 诊断结果:');
  console.log(` 平台: ${result.platform}`);
  console.log(` 权限: ${result.permissionGranted ? '已授予 ' : '未授予 '}`);
  console.log(` 可初始化: ${result.canInitializeAudioRecord ? '是 ' : '否 '}`);
  console.log(` 支持的采样率: ${result.supportedSampleRates.join(', ')} Hz`);

  if (result.nativeError) {
    console.log(` 原生错误: ${result.nativeError}`);
  }

  return result;
}

/**
 * 快速检查音频是否可用
 */
export async function isAudioAvailable(): Promise<boolean> {
  if (Platform.OS !== 'android') {
    return false;
  }

  try {
    const hasPermission = await PermissionsAndroid.check(
      PermissionsAndroid.PERMISSIONS.RECORD_AUDIO
    );

    if (!hasPermission) {
      return false;
    }

    // 快速测试默认采样率
    return new Promise((resolve) => {
      let resolved = false;
      const cleanup = () => {
        if (!resolved) {
          resolved = true;
          try {
            PCMStream.stopRecording();
          } catch (_e) {}
          errorListener.remove();
          clearTimeout(timeoutId);
        }
      };

      const errorListener = PCMStream.addListener('onError', () => {
        cleanup();
        resolve(false);
      });

      const timeoutId = setTimeout(() => {
        cleanup();
        resolve(true);
      }, 200);

      try {
```

```
        PCMStream.startRecording(48000, 1536, 16000);
      } catch (_e) {
        cleanup();
        resolve(false);
      }
    });
  } catch (_e) {
    return false;
  }
}

// ----------------------------------------

/**
 * 音频权限管理器
 * 自动检测和请求麦克风权限
 */

import { Platform, PermissionsAndroid, Alert, Linking } from 'react-native';

export interface PermissionStatus {
  granted: boolean;
  canAskAgain: boolean;
  shouldShowRationale: boolean;
}

/**
 * 检查麦克风权限状态
 */
export async function checkMicrophonePermission(): Promise<PermissionStatus> {
  if (Platform.OS !== 'android') {
    return {
      granted: true, // iOS 在 Info.plist 中声明，安装时自动授予
      canAskAgain: false,
      shouldShowRationale: false,
    };
  }

  try {
    const granted = await PermissionsAndroid.check(
      PermissionsAndroid.PERMISSIONS.RECORD_AUDIO
    );

    return {
      granted,
      canAskAgain: true, // Android 总是可以再次请求
      shouldShowRationale: !granted, // 如果未授予，应该显示说明
    };
  } catch (error) {
    console.error('检查权限失败:', error);
    return {
      granted: false,
      canAskAgain: false,
      shouldShowRationale: false,
    };
  }
}

/**
 * 请求麦克风权限（带友好提示）
 */
export async function requestMicrophonePermission(): Promise<boolean> {
  if (Platform.OS !== 'android') {
    return true;
  }

  try {
    // 先检查当前状态
    const status = await checkMicrophonePermission();

    if (status.granted) {
      console.log(' 麦克风权限已授予');
      return true;
    }

    // 请求权限
    console.log(' 请求麦克风权限...');
    const result = await PermissionsAndroid.request(
      PermissionsAndroid.PERMISSIONS.RECORD_AUDIO,
      {
        title: ' 需要麦克风权限',
        message: 'N.E.K.O. 需要访问您的麦克风来实现语音对话功能。',
        buttonNeutral: '稍后询问',
        buttonNegative: '取消',
```

```
      buttonPositive: '确定',
    }
  );

  switch (result) {
    case PermissionsAndroid.RESULTS.GRANTED:
      console.log(' 用户授予了麦克风权限');
      return true;

    case PermissionsAndroid.RESULTS.DENIED:
      console.log(' 用户拒绝了麦克风权限');
      Alert.alert(
        '权限被拒绝',
        '您拒绝了麦克风权限，语音功能将无法使用。您可以稍后在设置中授予权限。',
        [{ text: '知道了', style: 'default' }]
      );
      return false;

    case PermissionsAndroid.RESULTS.NEVER_ASK_AGAIN:
      console.log(' 用户选择"不再询问"');
      Alert.alert(
        '需要权限',
        '麦克风权限已被禁用。请在系统设置中手动授予麦克风权限。',
        [
          { text: '取消', style: 'cancel' },
          {
            text: '去设置',
            style: 'default',
            onPress: () => {
              openAppSettings();
            },
          },
        ]
      );
      return false;

    default:
      return false;
  }
} catch (error) {
  console.error(`请求权限失败:', error);
  return false;
}
}

/**
 * 打开应用设置页面
 */
export function openAppSettings(): void {
  if (Platform.OS === 'android') {
    try {
      // Android: 打开应用详情页
      Linking.openSettings();
    } catch (error) {
      console.error('无法打开设置:', error);
      Alert.alert('错误', '无法打开系统设置，请手动前往设置授予权限。');
    }
  } else if (Platform.OS === 'ios') {
    // iOS: 打开应用设置
    Linking.openSettings();
  }
}

/**
 * 确保麦克风权限（带重试）
 */
export async function ensureMicrophonePermission(
  maxRetries: number = 2
): Promise<boolean> {
  for (let attempt = 1; attempt <= maxRetries; attempt++) {
    console.log(` 尝试获取麦克风权限 (${attempt}/${maxRetries})`);

    const granted = await requestMicrophonePermission();

    if (granted) {
      return true;
    }

    // 如果不是最后一次尝试，等待一下
    if (attempt < maxRetries) {
      await new Promise((resolve) => setTimeout(resolve, 1000));
    }
  }
```

```
  return false;
}

// ----------------------------------------

export const AUDIO_PROCESSOR_CODE = `
// audio-processor.js
class AudioProcessor extends AudioWorkletProcessor {
  constructor(options) {
    super();

    // 获取采样率信息
    const processorOptions = options.processorOptions || {};
    this.originalSampleRate = processorOptions.originalSampleRate || 48000;
    this.targetSampleRate = processorOptions.targetSampleRate || 16000;

    // 计算重采样比率
    this.resampleRatio = this.targetSampleRate / this.originalSampleRate;

    // 创建缓冲区 - 使用较小的缓冲区以实现低延迟
    // 128帧在16kHz下约为8ms，在48kHz下约为2.7ms
    // 我们使用512帧的缓冲区，在16kHz下约为32ms
    this.bufferSize = 512;
    this.buffer = new Float32Array(this.bufferSize);
    this.bufferIndex = 0;

    // 用于重采样的临时缓冲区
    this.tempBuffer = [];

    console.log("AudioProcessor初始化: 原始采样率={originalSampleRate}Hz, 目标采样率={targetSampleRate}Hz");
  }

  process(inputs, outputs, parameters) {
    // 获取输入数据 (假设是单声道)
    const input = inputs[0][0];

    if (!input || input.length === 0) {
      return true;
    }

    // 将输入数据添加到临时缓冲区
    this.tempBuffer = this.tempBuffer.concat(Array.from(input));

    // 当临时缓冲区足够大时，执行重采样
    const requiredSamples = Math.ceil(this.bufferSize / this.resampleRatio);
    if (this.tempBuffer.length >= requiredSamples) {
      // 执行重采样
      const samplesNeeded = Math.min(requiredSamples, this.tempBuffer.length);
      const samplesToProcess = this.tempBuffer.slice(0, samplesNeeded);
      this.tempBuffer = this.tempBuffer.slice(samplesNeeded);

      // 执行重采样
      const resampledData = this.resampleAudio(samplesToProcess);

      // 将重采样后的数据转换为Int16Array
      const pcmData = new Int16Array(resampledData.length);
      for (let i = 0; i < resampledData.length; i++) {
        pcmData[i] = Math.max(-1, Math.min(1, resampledData[i])) * 0x7FFF;
      }

      // 发送到主线程
      this.port.postMessage(pcmData);
    }

    return true;
  }

  // 简单的线性插值重采样
  resampleAudio(audioData) {
    const inputLength = audioData.length;
    const outputLength = Math.floor(inputLength * this.resampleRatio);
    const result = new Float32Array(outputLength);

    for (let i = 0; i < outputLength; i++) {
      const position = i / this.resampleRatio;
      const index = Math.floor(position);
      const fraction = position - index;

      // 线性插值
      if (index + 1 < inputLength) {
        result[i] = audioData[index] * (1 - fraction) + audioData[index + 1] * fraction;
      } else {
        result[i] = audioData[index];
      }
```

```
    }

    return result;
  }
}

registerProcessor('audio-processor', AudioProcessor);

`;

// ----------------------------------------

// Android 环境检查脚本
// 在 React Native WebView 中运行此脚本来检查环境兼容性

const checkAndroidEnvironment = () => {
  const results = {
    userAgent: navigator.userAgent,
    webViewVersion: null,
    androidVersion: null,
    apiLevel: null,
    audioWorkletSupport: false,
    secureContext: false,
    recommendations: []
  };

  // 解析 User Agent
  const ua = navigator.userAgent;
  console.log('User Agent:', ua);

  // 检查 Android 版本
  const androidMatch = ua.match(/Android (\d+(?:\.\d+)?)/);
  if (androidMatch) {
    results.androidVersion = androidMatch[1];
    const version = parseFloat(androidMatch[1]);

    // 估算 API Level
    if (version >= 14) results.apiLevel = 34;
    else if (version >= 13) results.apiLevel = 33;
    else if (version >= 12) results.apiLevel = 31;
    else if (version >= 11) results.apiLevel = 30;
    else if (version >= 10) results.apiLevel = 29;
    else if (version >= 9) results.apiLevel = 28;
    else if (version >= 8.1) results.apiLevel = 27;
    else if (version >= 8.0) results.apiLevel = 26;
    else if (version >= 7.1) results.apiLevel = 25;
    else if (version >= 7.0) results.apiLevel = 24;
  }

  // 检查 WebView 版本
  const webViewMatch = ua.match(/Chrome\/(\d+)/);
  if (webViewMatch) {
    results.webViewVersion = parseInt(webViewMatch[1]);
  }

  // 检查 AudioWorklet 支持
  try {
    const AudioContextClass = window.AudioContext || window.webkitAudioContext;
    if (AudioContextClass) {
      const testContext = new AudioContextClass();
      results.audioWorkletSupport = !!testContext.audioWorklet;
      testContext.close();
    }
  } catch (e) {
    console.error('AudioContext test failed:', e);
  }

  // 检查安全上下文
  results.secureContext = window.isSecureContext;

  // 生成建议
  if (!results.audioWorkletSupport) {
    if (results.apiLevel < 29) {
      results.recommendations.push('升级到 Android 10+ (API Level 29+) 以获得 AudioWorklet 支持');
    }
    if (results.webViewVersion && results.webViewVersion < 66) {
      results.recommendations.push('WebView 版本过低，需要 Chrome 66+ 版本');
    }
    if (!results.secureContext) {
      results.recommendations.push('使用 HTTPS 连接以获得完整功能支持');
    }
  }

  // 添加通用建议
```

```
  if (results.apiLevel < 33) {
    results.recommendations.push('建议使用 Android 13+ (API Level 33+) 以获得最佳兼容性');
  }

  return results;
};

// 在 WebView 中运行检查
if (typeof window !== 'undefined') {
  const envCheck = checkAndroidEnvironment();
  console.log('=== Android 环境检查结果 ===');
  console.log(envCheck);

  // 发送结果到 React Native
  if (window.ReactNativeWebView) {
    window.ReactNativeWebView.postMessage(JSON.stringify({
      type: 'environment_check',
      data: envCheck
    }));
  }
}

export default checkAndroidEnvironment;

// ----------------------------------------

export type DevConnectionConfig = {
  host: string;
  port: number;
  characterName: string;
  // P2P 连接配置（v3: 两层回退）
  p2p?: {
    token: string;
    deviceId?: string;          // 设备 ID（用于云端查询）

    // 第1层：LAN 直连
    lanIp?: string;             // 局域网 IP
    lanPort?: number;           // 局域网端口

    // 第2层：STUN 打洞
    stunIp?: string;            // STUN 公网 IP
    stunPort?: number;          // STUN 公网端口
  };
};

export const DEFAULT_DEV_CONNECTION_CONFIG: DevConnectionConfig = {
  host: process.env.EXPO_PUBLIC_DEV_HOST || '192.168.1.8',
  port: Number(process.env.EXPO_PUBLIC_DEV_PORT) || 48911,
  characterName: process.env.EXPO_PUBLIC_DEV_CHARACTER || 'test',
};

export function parseDevConnectionConfig(raw: string): Partial<DevConnectionConfig> | null {
  const text = raw.trim();
  if (!text) return null;

  // 1) JSON: {"host":"x","port":48911,"characterName":"test"}
  //    或 P2P 格式: {"lan_ip":"x","port":48920,"token":"xxx"}
  try {
    const obj = JSON.parse(text) as any;
    if (obj && typeof obj === 'object') {
      const out: Partial<DevConnectionConfig> = {};

      // 检测 P2P 格式（包含 lan_ip 和 token）- v3 架构：两层回退
      if (typeof obj.lan_ip === 'string' && obj.lan_ip.trim() && typeof obj.token === 'string') {
        out.host = obj.lan_ip.trim();
        out.port = typeof obj.port === 'number' ? obj.port : 48920;
        out.characterName = obj.character || obj.name || 'test';

        // v3: 完整的两层连接信息
        out.p2p = {
          token: obj.token,
          deviceId: obj.device_id,

          // 第1层：LAN 直连
          lanIp: obj.lan_ip.trim(),
          lanPort: obj.port || 48920,

          // 第2层：STUN 打洞
          stunIp: obj.stun_ip,
          stunPort: obj.stun_port,
        };
        return out;
      }
```

```
      // 普通格式
      if (typeof obj.host === 'string' && obj.host.trim()) out.host = obj.host.trim();
      if (typeof obj.port === 'number' && Number.isFinite(obj.port)) out.port = obj.port;
      if (typeof obj.characterName === 'string' && obj.characterName.trim()) out.characterName = obj.characterName.trim();
      if (typeof obj.name === 'string' && obj.name.trim()) out.characterName = obj.name.trim();
      if (Object.keys(out).length > 0) return out;
    }
  } catch {
    // ignore
  }

  // 2) URL-like: nekorn://dev?host=...&port=...&name=...
  try {
    const url = new URL(text);
    const host = (url.searchParams.get('host') || '').trim();
    const portStr = (url.searchParams.get('port') || '').trim();
    const name = (url.searchParams.get('characterName') || url.searchParams.get('name') || '').trim();
    const out: Partial<DevConnectionConfig> = {};
    if (host) out.host = host;
    if (portStr && /^\d+$/.test(portStr)) out.port = Number(portStr);
    if (name) out.characterName = name;

    // 允许直接从 URL 的 host/port 取值（如 http://1.2.3.4:48911）
    if (!out.host && url.hostname) out.host = url.hostname;
    if (out.port == null && url.port && /^\d+$/.test(url.port)) out.port = Number(url.port);

    if (Object.keys(out).length > 0) return out;
  } catch {
    // ignore
  }

  // 3) host:port 或 host:port?name=xxx
  // 允许 ws:// / http:// 前缀在这里被粗略剥离
  const stripped = text.replace(/^(ws|wss|http|https):\/\//, '');
  const parts = stripped.split('?');
  const hostPort = (parts[0] || '').trim();
  const query = (parts[1] || '').trim();
  const m = hostPort.match(/^([a-zA-Z0-9.\-]+)(?::(\d+))?$/);
  if (m) {
    const out: Partial<DevConnectionConfig> = {};
    if (m[1]) out.host = m[1];
    if (m[2]) out.port = Number(m[2]);
    if (query) {
      try {
        const q = new URLSearchParams(query);
        const name = (q.get('characterName') || q.get('name') || '').trim();
        if (name) out.characterName = name;
      } catch {
        // ignore
      }
    }
    if (Object.keys(out).length > 0) return out;
  }

  return null;
}

export function buildHttpBaseURL(config: Pick<DevConnectionConfig, 'host' | 'port'>): string {
  const host = String(config.host || '').trim();
  const port = Number(config.port);
  return `http://${host}:${port}`;
}

/**
 * 给任意 URL 追加 P2P token query 参数。
 * 非 P2P 模式（token 为空）时原样返回。
 */
export function appendP2PToken(url: string, token?: string): string {
  if (!token) return url;
  const sep = url.includes('?') ? '&' : '?';
  return `${url}${sep}token=${token}`;
}

// ----------------------------------------

export const ERROR_CATCH_CODE = `
(function () {
    //  立即设置错误捕获，在任何其他代码执行之前
    window.addEventListener('error', function (event) {
        var errorInfo = {
            message: event.message,
            filename: event.filename,
            lineno: event.lineno,
            colno: event.colno,
```

```
        error: event.error ? event.error.toString() : '未知错误',
        stack: event.error ? event.error.stack : '无堆栈信息',
        timestamp: new Date().toISOString()
      };

      console.error(' WebView JavaScript 全局错误:', errorInfo);

      if (window.ReactNativeWebView) {
        window.ReactNativeWebView.postMessage(JSON.stringify({
          type: 'javascript_error',
          error: errorInfo,
          source: 'webview_early_capture'
        }));
      }
    }, true); // 使用捕获阶段，确保优先处理

    // 捕获未处理的 Promise 拒绝
    window.addEventListener('unhandledrejection', function (event) {
      var rejectionInfo = {
        reason: event.reason ? event.reason.toString() : '未知原因',
        stack: event.reason && event.reason.stack ? event.reason.stack : '无堆栈信息',
        timestamp: new Date().toISOString()
      };

      console.error(' WebView Promise 拒绝:', rejectionInfo);

      if (window.ReactNativeWebView) {
        window.ReactNativeWebView.postMessage(JSON.stringify({
          type: 'promise_rejection',
          error: rejectionInfo,
          source: 'webview_early_capture'
        }));
      }
    });

    // 监听资源加载错误
    window.addEventListener('error', function (event) {
      if (event.target !== window && event.target) {
        var resourceError = {
          type: 'resource_error',
          tagName: event.target.tagName,
          src: event.target.src || event.target.href || '未知资源',
          message: '资源加载失败',
          timestamp: new Date().toISOString()
        };

        console.error(' WebView 资源加载错误:', resourceError);

        if (window.ReactNativeWebView) {
          window.ReactNativeWebView.postMessage(JSON.stringify({
            type: 'resource_error',
            error: resourceError,
            source: 'webview_early_capture'
          }));
        }
      }
    }, true);

    console.log(' WebView 错误捕获机制已设置');
    console.log(' 注入的JavaScript开始执行');
    console.log(' 检查ReactNativeWebView接口:', !!window.ReactNativeWebView);
})();
`;

// ----------------------------------------

import { Directory, File, Paths } from 'expo-file-system';

function resolveUrl(baseUrl: string, relativePath: string): string {
  try {
    const resolved = new URL(relativePath, baseUrl);
    // 保留 base URL 的 query 参数（如 P2P token），new URL() 会丢弃它们
    const base = new URL(baseUrl);
    base.searchParams.forEach((v, k) => {
      if (!resolved.searchParams.has(k)) resolved.searchParams.set(k, v);
    });
    return resolved.toString();
  } catch {
    if (baseUrl.endsWith('/')) return baseUrl + relativePath;
    return baseUrl + '/' + relativePath;
  }
}

async function ensureDirAsync(dir: string): Promise<void> {
```

```
  try {
    const destination = new Directory(dir);
    if (!destination.exists) {
      destination.create();
    }
  } catch (error) {
    console.log('ensureDirAsync error: ', error)
  }
}

async function downloadFileTo(dstPath: string, srcUrl: string): Promise<void> {
  console.log(`loadModelFile ${srcUrl} => ${dstPath}`)
  const dstDir = dstPath.substring(0, dstPath.lastIndexOf('/') + 1);
  await ensureDirAsync(dstDir);
  try {
    // 提取文件名（处理包含子目录的相对路径，如 Nahida_1080.1024/texture_00.png）
    const lastSlashIndex = dstPath.lastIndexOf('/');
    const fileName = dstPath.substring(lastSlashIndex + 1);
    const expectedUri = `${dstDir}${fileName}`;

    // 如果目标文件已存在，跳过下载
    const existingFile = new File(expectedUri);
    if (existingFile.exists) {
      console.log(`skip (exists): ${expectedUri}`);
      return;
    }

    // downloadFileAsync 用 URL 末尾的文件名作为目标，先清理可能残留的同名文件
    const urlFileName = srcUrl.split('?')[0].split('/').pop() ?? fileName;
    const downloadTarget = new File(`${dstDir}${urlFileName}`);
    if (downloadTarget.exists) {
      await downloadTarget.delete();
    }

    const dstFolder = new Directory(dstDir);
    // downloadFileAsync 返回 File 对象，有 .uri 属性
    const downloaded = await File.downloadFileAsync(srcUrl, dstFolder);
    const downloadedUri = downloaded.uri;

    // 如果下载的文件名与目标文件名不同（URL encoded vs decoded），需要重命名
    const normalizedDownloaded = (() => { try { return decodeURIComponent(downloadedUri); } catch { return downloadedUri; } })();
    const normalizedExpected = (() => { try { return decodeURIComponent(expectedUri); } catch { return expectedUri; } })();

    if (normalizedDownloaded !== normalizedExpected) {
      const downloadedFile = new File(downloadedUri);
      const targetFile = new File(expectedUri);
      if (targetFile.exists) {
        targetFile.delete();
      }
      downloadedFile.move(targetFile);
      console.log(`moved file: ${downloadedUri} => ${expectedUri}`);
    } else {
      console.log(`loaded file: ${downloadedUri}`);
    }
  } catch (error) {
    console.error(` downloadFileTo failed: ${dstPath}`, error)
  }
}

export async function removeDownloadedModel(targetDirName: string): Promise<void> {
  const destination = new Directory(Paths.cache, targetDirName);
  console.log('removeDownloadedModel: ', destination.uri);

  if (await destination.exists) {
    console.log('removeDownloadedModel destination.uri: ', destination.uri);

    try {
      // 确保先清空目录
      const files = await destination.list();
      for (const file of files) {
        await file.delete();
      }

      await destination.delete(); // 注意加 await
      console.log("Deleted successfully:", destination.uri);
    } catch (err) {
      console.error("Failed to delete:", destination.uri, err);
    }
  }
}

/**
 * 已经把 model3.json 下载到本地后，继续解析并下载所有依赖到同一目录结构。
 * @param localModelJsonUri 例如: file:///data/user/0/<pkg>/cache/live2d/mao_pro/mao_pro.model3.json
```

```
 * @param remoteBaseUrl 远端基地址，例如: https://example.com/live2d/mao_pro/
 * @returns 返回原始的 localModelJsonUri
 */
export async function downloadDependenciesFromLocalModel(
  localModelJsonUri: string,
  remoteBaseUrl: string
): Promise<string> {
  // 读取本地 model3.json
  const file = new File(localModelJsonUri);

  const modelJsonStr = file.textSync()
  const model = JSON.parse(modelJsonStr);

  console.log('downloadDependenciesFromLocalModel model: ', model)

  // 计算本地根目录（保持与 model3.json 同一根目录）
  // e.g. file:///.../live2d/mao_pro/mao_pro.model3.json -> file:///.../live2d/mao_pro/
  const lastSlash = localModelJsonUri.lastIndexOf('/') + 1;
  const targetRoot = localModelJsonUri.substring(0, lastSlash);

  const files: string[] = [];
  if (model.FileReferences) {
    const refs = model.FileReferences;
    if (refs.Moc) files.push(refs.Moc);
    if (refs.Textures) files.push(...refs.Textures);
    if (refs.Physics) files.push(refs.Physics);
    if (refs.Pose) files.push(refs.Pose);
    if (refs.DisplayInfo) files.push(refs.DisplayInfo);
    if (refs.Expressions) files.push(...refs.Expressions.map((e: any) => e.File));
    if (refs.Motions) {
      Object.values(refs.Motions).forEach((arr: unknown) => {
        (arr as any[]).forEach((m) => {
          if (m && m.File) files.push(m.File);
        });
      });
    }
  }

  const uniqueFiles = Array.from(new Set(files.filter(Boolean)));
  console.log(' 需要下载的依赖文件:', uniqueFiles);

  // 串行下载，避免并发时同名文件冲突
  for (const relPath of uniqueFiles) {
    const src = resolveUrl(remoteBaseUrl, relPath);
    const dst = `${targetRoot}${relPath}`;
    await downloadFileTo(dst, src);
  }

  return localModelJsonUri;
}



// ----------------------------------------

import { PermissionsAndroid, Platform, Alert, Linking } from 'react-native';
import { PERMISSIONS, request, RESULTS } from 'react-native-permissions';

/**
 * 请求麦克风权限（旧版本，保持向后兼容）
 * @deprecated 使用 audioPermissionManager 中的函数代替
 */
export const requestMicrophonePermission = async () => {
  if (Platform.OS === 'ios') {
    const result = await request(PERMISSIONS.IOS.MICROPHONE);
    return result === RESULTS.GRANTED;
  } else {
    // Android: 增强权限请求
    try {
      // 先检查权限状态
      const checkResult = await PermissionsAndroid.check(
        PermissionsAndroid.PERMISSIONS.RECORD_AUDIO
      );

      if (checkResult) {
        console.log(' 麦克风权限已授予');
        return true;
      }

      // 请求权限（带友好提示）
      const granted = await PermissionsAndroid.request(
        PermissionsAndroid.PERMISSIONS.RECORD_AUDIO,
        {
          title: ' 需要麦克风权限',
```

```
        message: 'N.E.K.O. 需要访问您的麦克风来实现语音对话功能。',
        buttonNeutral: '稍后询问',
        buttonNegative: '取消',
        buttonPositive: '确定',
      }
    );

    if (granted === PermissionsAndroid.RESULTS.GRANTED) {
      console.log(' 用户授予了麦克风权限');
      return true;
    } else if (granted === PermissionsAndroid.RESULTS.NEVER_ASK_AGAIN) {
      console.log(' 用户选择"不再询问"');
      // 修复：延迟显示对话框，避免自动跳转
      setTimeout(() => {
        Alert.alert(
          '需要麦克风权限',
          '您之前拒绝了麦克风权限并选择了"不再询问"。\n\n语音功能需要麦克风权限才能工作。是否前往设置开启权限？',
          [
            {
              text: '暂不开启',
              style: 'cancel',
              onPress: () => console.log('用户选择不去设置'),
            },
            {
              text: '去设置',
              style: 'default',
              onPress: () => {
                try {
                  Linking.openSettings();
                } catch (error) {
                  console.error('无法打开设置:', error);
                  Alert.alert('错误', '无法打开系统设置，请手动前往设置授予权限。');
                }
              },
            },
          ],
          { cancelable: true }
        );
      }, 500); // 延迟 500ms
      return false;
    } else {
      console.log(' 用户拒绝了麦克风权限');
      // 修复：拒绝时也显示提示，但不要跳转
      setTimeout(() => {
        Alert.alert(
          '权限被拒绝',
          '您拒绝了麦克风权限，语音功能将无法使用。\n\n您可以稍后在设置中手动授予权限。',
          [{ text: '知道了', style: 'default' }],
          { cancelable: true }
        );
      }, 300);
      return false;
    }
  } catch (error) {
    console.error('请求麦克风权限失败:', error);
    setTimeout(() => {
      Alert.alert('错误', '请求权限时发生错误，请稍后重试。');
    }, 300);
    return false;
  }
}
};

// ----------------------------------------

// Web platform permissions utility
export const requestMicrophonePermission = async () => {
  try {
    if (navigator.mediaDevices && navigator.mediaDevices.getUserMedia) {
      // Request microphone permission by attempting to access the media stream
      const stream = await navigator.mediaDevices.getUserMedia({
        audio: {
          sampleRate: 48000,
          channelCount: 1,
          echoCancellation: true,
          noiseSuppression: true,
          autoGainControl: true
        }
      });

      // Immediately stop the stream as we only need permission
      stream.getTracks().forEach(track => track.stop());
      return true;
    }
```

```
    return false;
  } catch (error) {
    console.error('Failed to request microphone permission:', error);
    return false;
  }
};

// ----------------------------------------

/**
 * 全局 WebSocket 会话状态（纯内存，App 重启自动重置为 false）。
 * main.tsx 在 WebSocket 连接/断开时调用 set()；
 * 其他页面（如 index.tsx）通过 subscribe() 被动监听。
 */
type Listener = (connected: boolean) => void;

let _connected = false;
const _listeners = new Set<Listener>();

export const sessionStore = {
  get isConnected(): boolean {
    return _connected;
  },

  set(connected: boolean): void {
    if (_connected === connected) return;
    _connected = connected;
    _listeners.forEach((l) => { l(connected); });
  },

  subscribe(listener: Listener): () => void {
    _listeners.add(listener);
    return () => _listeners.delete(listener);
  },
};

// ----------------------------------------

export const WRAP_CONSOLE_METHOD_CODE = `
(function () {
  if (window.__RN_LOG_BRIDGE__) return;
  window.__RN_LOG_BRIDGE__ = true;

  function safeStringify(obj) {
    const seen = new WeakSet();
    return JSON.stringify(obj, (key, value) => {
      if (typeof value === "object" && value !== null) {
        if (seen.has(value)) return "[Circular]";
        seen.add(value);
      }
      return value;
    });
  }

  function initLogBridge() {
    try { window.ReactNativeWebView && window.ReactNativeWebView.postMessage("initLogBridge"); } catch(e) {}
    if (!(window.ReactNativeWebView && typeof window.ReactNativeWebView.postMessage === 'function')) {
      setTimeout(initLogBridge, 500);
      return;
    }

    function sendToReactNative(type, args) {
      try {
        const message = args.map(arg =>
          typeof arg === 'object' ? safeStringify(arg) : String(arg)
        ).join(' ');
        window.ReactNativeWebView.postMessage(JSON.stringify({ type, message }));
      } catch (e) {}
    }

    const methods = ['log', 'warn', 'error', 'info', 'debug'];
    methods.forEach(fn => {
      const original = console[fn] || function () {};
      console[fn] = function (...args) {
        sendToReactNative(fn, args);
        try { original(...args); } catch (e) {}
      };
    });

    try { console.log('system', ' ReactNative WebView 日志桥接就绪'); } catch(e) {}
  }

  initLogBridge();
})();
```